



南京航空航天大学

NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

机电学院



先进液压与机电控制实验室

Advanced Hydraulic and Electromechanical Control

# 可编程控制器

主讲：凌杰

机械电子工程系

Email: [meejling@nuaa.edu.cn](mailto:meejling@nuaa.edu.cn)

NUAA



## 第3章 CP1 PLC的指令系统



## 第3章 CP1 PLC的指令系统

❖ CP1H PLC的指令系统由基本指令和高级指令组成。

- **基本指令**包括：输入、输出、定时、计数
- **高级指令**包括：比较类指令、传送类指令、移位类指令、转换类指令、运算类指令、通信类指令等。



## 第3章 CP1 PLC的指令系统

### 3.1 概述

### 3.2 基本指令

### 3.3 顺序控制指令

### 3.4 定时器/计数器指令

### 3.5 数据传送指令

### 3.6 数据比较指令

### 3.7 数据移位指令

### 3.8 数据转换指令

### 3.9 递增/递减指令

### 3.10 四则运算指令

### 3.11 逻辑运算指令

### 3.12 单精度浮点转换与运算指令

### 3.13 子程序控制指令

### 3.14 高速计数/脉冲输出指令

### 3.15 步进指令

### 3.16 中断控制指令



## 3.1 概述

### 1. 指令的格式

格式:

指令的功能

指令的代码, 用三位数字表示

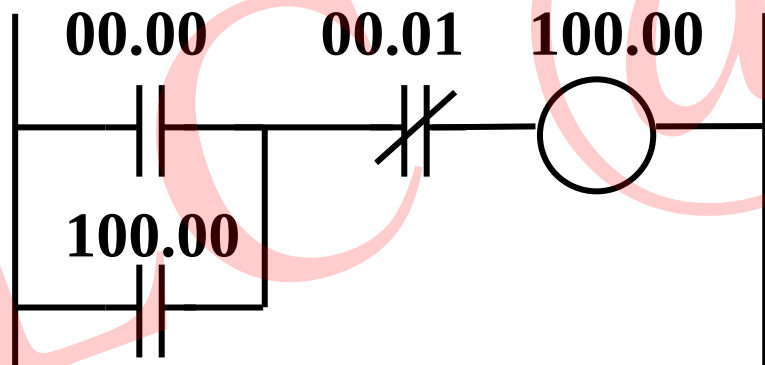
指令执行的对象, 个数由指令决定

助记符 (指令码)

操作数1

操作数2

操作数3



梯形图程序

|         |        |
|---------|--------|
| LD      | 00.00  |
| OR      | 100.00 |
| AND NOT | 00.01  |
| OUT     | 100.00 |

语句表程序



## 3.1 概述

### 2. 操作数形式

#### 1) 继电器号

继电器区CIO、W、H、A、TR、T、C中的继电器编号。CIO区中被输入点占用的继电器不要用作程序的输出，即不要用程序去改变其状态。

#### 2) 通道（字）号

通道的编号；CIO区中的通道号没有前缀，注意与常数区分，例如SV

计数器指令：   CNT   0000  
                                  SV



## 3.1 概述

### 2. 操作数形式

#### 3) DM间接寻址

◆ DM区可以采用BCD模式或BIN模式进行间接寻址。

① BCD模式寻址（\*D）。若DM区地址前缀一个“\*”字符，则DM字中的内容将按BCD数处理，指令将在此BCD数所指的DM字上进行操作，只有部分DM区（D00000～D09999）可以通过BCD数000～9999进行间接寻址。

|          |
|----------|
| MOV(021) |
| #3560    |
| *D00010  |

BCD 模式间接寻址



## 3.1 概述

### 2. 操作数形式

#### 3) DM间接寻址

② 二进制模式寻址（@D）。若DM区地址前缀一个“@”字符，则DM字中的内容将按无符号二进制数处理，指令将在此二进制所指的DM字上进行操作，全部DM区都可以通过十六进制数0000~7FFFH进行间接寻址。

|          |
|----------|
| MOV(021) |
| #3560    |
| @D00010  |

二进制模式间接寻址





## 3.1 概述

### 2. 操作数形式

#### 4) 常数

- 操作数为常数时，可以是十进制，也可以是十六进制，这取决于指令的需要
- 16位单通道常数有四种形式

| 数据形式    | 标识 | 范围            |
|---------|----|---------------|
| 无符号 BIN | #  | #0000~#FFFF   |
| 带符号十进制数 | ±  | -32768~+32767 |
| 无符号十进制数 | &  | &0~&65535     |
| BCD 数   | #  | #0~#9999      |

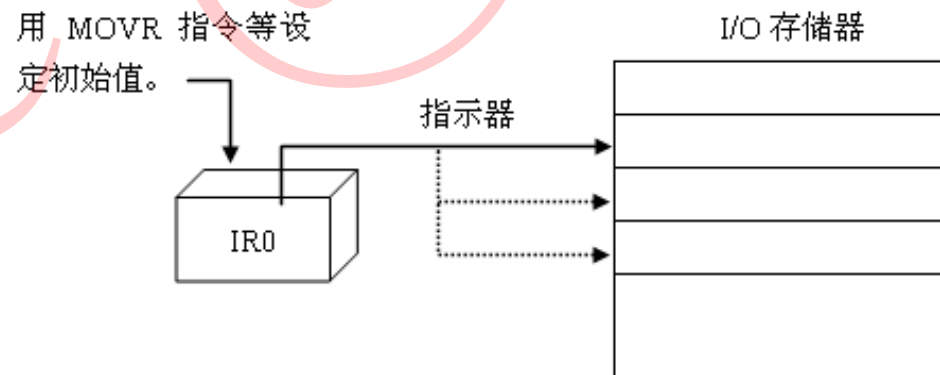


## 3.1 概述

### 2. 操作数形式

#### 5) 索引寄存器IR间接寻址

- 共有16个索引寄存器：IR0~IR15
- 每个索引寄存器32位，存储PLC内存区中一个位或字的绝对地址，可用于对内存区中的位或字进行间接寻址
- 用MOVR指令将一个常规数据区地址转换成它的PLC存储地址，并将该值写到指定的IR中（用MOVW指令在IR中设定定时器/计数器当前值的PLC存储地址）



索引寄存器工作原理



## 3.1 概述

### 2. 操作数形式

#### 5) 索引寄存器IR间接寻址

- 若IR带前缀“，”作为操作数，则指令将在IR中PLC存储地址所指的字上进行操作，而不是索引寄存器，IR即为I/O存储区的**指针**
- I/O存储区（除索引寄存器、数据寄存器和状态标志位以外）中所有的地址都能用PLC存储地址惟一指定，无须指定数据区
- 除了基本的间接寻址外，还可以用常数，数据寄存器及自动增加或减少偏移IR中的PLC存储地址等方式实现每次执行指令时增大或减少地址来循环读写数据。



## 3.1 概述

### 2. 操作数形式

#### 5) 索引寄存器IR间接寻址

IR 间接寻址变量表

| 变 量         | 功 能                                                   | 句 法                     | 示 例                                                 |
|-------------|-------------------------------------------------------|-------------------------|-----------------------------------------------------|
| 间接寻址        | IR□的内容作为一个位或字的 PLC 存储地址处理                             | ,IR□                    | LD ,IR0<br>装载 IR0 所含的 PLC 存储地址位                     |
| 常量偏移间接寻址    | IR□的内容加上常量前缀所得的值作为一个字或位的 PLC 存储地址处理,常数为-2048~2047 的整数 | 常数,IR□<br>(正负数均可)       | LD +5,IR0<br>IR0 的内容加 5 作为 PLC 存储地址,并装载该地址中的位       |
| DR 偏移间接寻址   | IR□的内容加上 DR 的内容所得的值作为一个字或位的 PLC 存储地址                  | DR□,IR□                 | LD DR0,IR0<br>IR0 的内容加 DR0 的内容作为 PLC 存储地址,并装载该地址中的位 |
| 地址自动递增的间接寻址 | IR□的内容作为一个位或字的 PLC 存储地址后,IR□的内容自动加 1 或 2              | 加 1: IR□+<br>加 2: IR□++ | LD ,IR0++<br>装载 IR0 中 PLC 存储地址中的位,并将 IR0 的内容加上 2    |
| 地址自动递减的间接寻址 | IR□的内容自动减 1 或 2 并将结果作为一个字或位的 PLC 存储地址                 | 减 1: -IR□<br>减 2: --IR□ | LD ,--IR0<br>IR0 的内容减 2 作为 PLC 存储地址并装载该地址中的位        |



## 3.1 概述

### 2. 操作数形式

#### 5) 索引寄存器IR间接寻址

- 将字CIO 00002的PLC存储地址存储到索引寄存器IR0中

MOV R 2 将 CIO00002 的 PLC 存储地址存储在 IR0 中

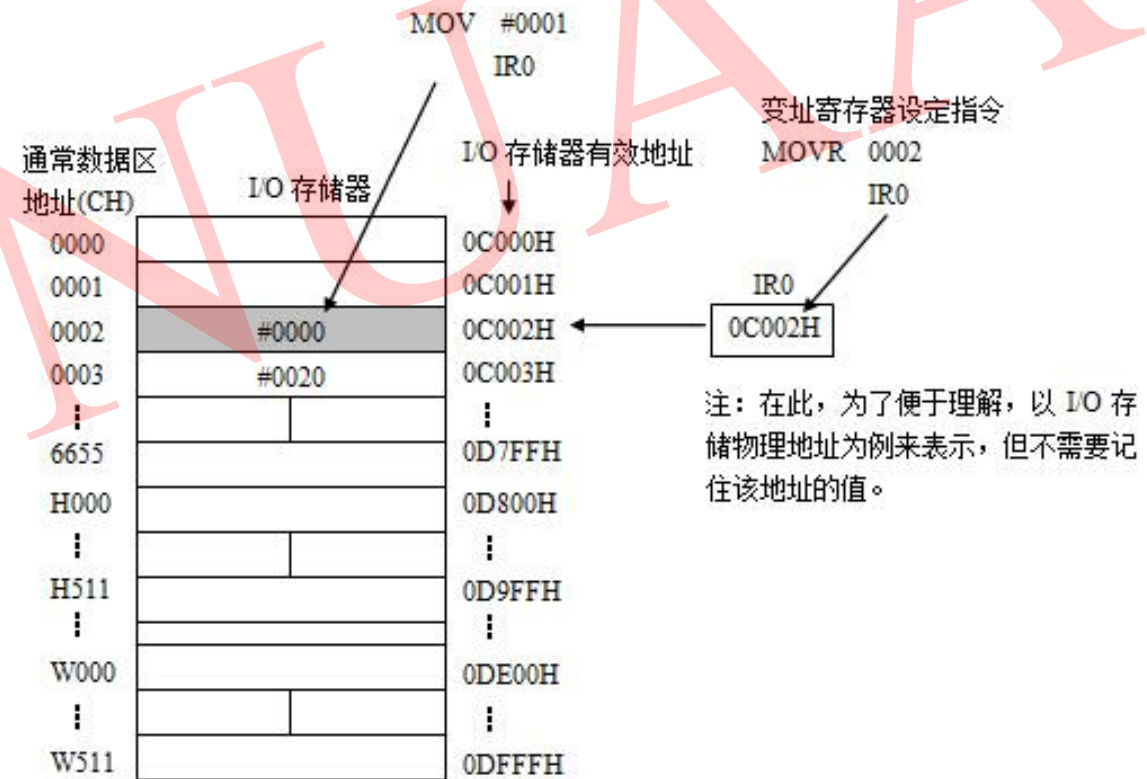
IR0

MOV #0001 将#0001 写到 IR0 所含的 PLC 存储地址中

,IR0

MOV #0020 读取 IR0 的内容并加 1 作为 PLC 存储地址, 将#0020 写到该地址中

+1,IR0



IR 间接寻址示例



## 3.1 概述

### 2. 操作数形式

#### 6) 索引寄存器IR直接寻址

□ 若IR不带前缀“，”作为操作数时，指令将IR本身的内容（双字）进行操作，表列出了可对IR直接寻址的指令，当这些指令对索引寄存器操作时，后者作为指针。

适用 IR 直接寻址指令

| 指令组       | 指令名称                   | 助记符   |
|-----------|------------------------|-------|
| 数据传送指令    | 索引寄存器设置                | MOVR  |
|           | 索引寄存器设置（定时器/计数器当前值存储器） | MOVRL |
|           | 双字传送                   | MOVL  |
|           | 双字数据交换                 | XCGL  |
| 表格数据处理指令  | 记录位置设置                 | SETR  |
|           | 记录号读取                  | GETR  |
| 递增/递减指令   | 二进制递增                  | ++L   |
|           | 二进制递减                  | --L   |
| 比较指令      | 双字等于                   | =L    |
|           | 双字不等于                  | <>L   |
|           | 双字小于                   | <L    |
|           | 双字小于或等于                | <=L   |
|           | 双字大于                   | >L    |
|           | 双字大于或等于                | >=L   |
|           | 双字比较                   | CMPL  |
| 带符号算术运算指令 | 带符号双字无进位二进制加法          | +L    |
|           | 带符号双字无进位二进制减法          | -L    |

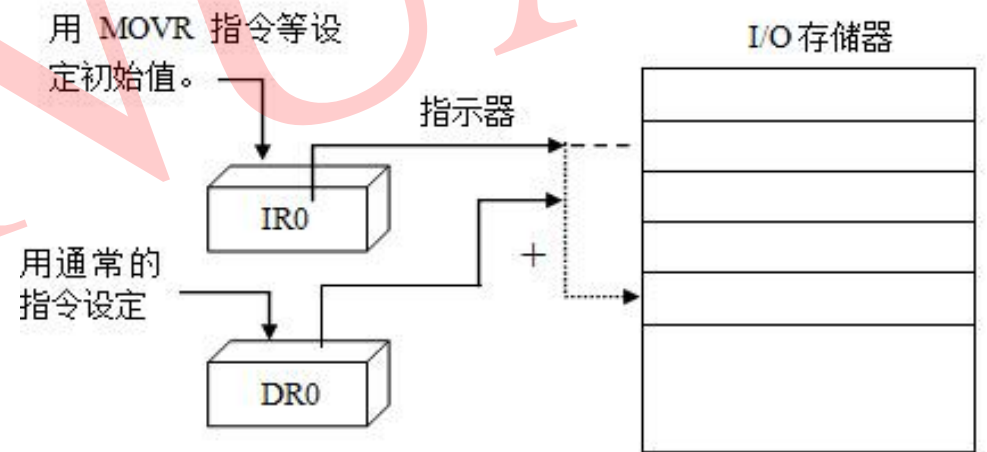


## 3.1 概述

### 2. 操作数形式

#### 7) 数据寄存器DR

- 共有16个数据寄存器：DR0~DR15
- 利用数据寄存器来偏移索引寄存器中的PLC存储地址。将数据寄存器中的值加到索引寄存器中的PLC存储地址上，来指定一个位或字在I/O存储区中的绝对内存地址
- 数据寄存器中的数据是带符号的二进制数，取值范围是-32768~+32767，因此索引寄存器中的内容既可以偏移到高地址，也可以偏移到低地址。



数据寄存器工作原理



## 3.1 概述

### 2. 操作数形式

#### 7) 数据寄存器DR

□ 用数据寄存器来偏移索引寄存器中的PLC存储地址

|     |         |                                              |
|-----|---------|----------------------------------------------|
| LD  | DR0,IR0 | 将 DR0、IR0 的内容相加得到 PLC 存储地址，将该地址的位装载到 PLC 内存中 |
| MOV | #0001   | 将 DR0、IR1 的内容相加得到 PLC 存储地址，并将#0001 写到该地址中    |
|     | DR0,IR1 |                                              |





## 3.1 概述

### 3. 标志位

- ❑ 错误标志ER：是用于监视指令执行的最常用的标志，ON时，表明正在执行的当前指令出错，停止执行。
- ❑ 等于标志EQ：在比较指令中的两个操作数相等或指令计算结果为0时，该标志置ON。
- ❑ 负标志N：在指令计算结果的最高有效位（符号位）为1时，该标志置ON。

指令执行可能影响的标志位，见后表。



## 状态标志功能位

| 名 称    | 标记  | 符号    | 功 能                                                                                                                             |
|--------|-----|-------|---------------------------------------------------------------------------------------------------------------------------------|
| 错误标志   | ER  | P_ER  | 指令中的操作数据不正确（指令处理错误）时，该标志置 ON，以指使指令因错误而停止。如果在“PLC 设置”中设置为停止指令错误时停止操作（指令操作错误），则在错误标志置 ON 时停止程序的执行，并且指令处理错误标志（A429510）也置 ON。       |
| 存取错误标志 | AER | P_AER | 出现非法存取错误时，该标志置 ON。非法存取错误指示某指令试图访问一个不应该被访问的存储区域。如果在“PLC 设置”中设置为指令错误（指令操作错误）时停止操作，则在存取错误标志置 ON 时停止程序的执行，并且指令处理错误标志（A429510）也置 ON。 |
| 进位标志   | CY  | P_CY  | 算术运算结果中出现进位或数据移位指令将一个“1”移进进位标志时，该标志置 ON。进位标志是某些算术运算和符号运算指令执行结果的一部分。                                                             |
| 大于标志   | >   | P_GT  | 当比较指令中的第一个操作数大于第二个操作数或一个值大于指定范围时，该标志置 ON。                                                                                       |
| 等于标志   | =   | P_EQ  | 当比较指令中的两个操作数相等或某一计算结果为 0 时，该标志置 ON。                                                                                             |
| 小于标志   | <   | P_LT  | 当比较指令中的第一个操作数小于第二个操作数或一个值小于指定范围时，该标志置 ON。                                                                                       |
| 负标志    | N   | P_N   | 当结果的最高有效位（符号位）为 ON 时，该标志置 ON。                                                                                                   |
| 上溢标志   | OF  | P_OF  | 当计算结果溢出结果字容量的上限时，该标志置 ON。                                                                                                       |
| 下溢标志   | UF  | P_UF  | 当计算结果溢出结果字容量的下限时，该标志置 ON。                                                                                                       |
| 大于等于标志 | >=  | P_GE  | 当比较指令中的第一个操作数大于或等于第二个操作数时，该标志置 ON。                                                                                              |
| 不等于标志  | <>  | P_NE  | 当比较指令中的两个操作数不相等时，该标志置 ON。                                                                                                       |
| 小于等于标志 | <=  | P_LE  | 当比较指令中的第一个操作数大于或等于第二个操作数时，该标志置 ON。                                                                                              |
| 常通标志   | ON  | P_ON  | 该标志总是为 ON（总是 1）。                                                                                                                |
| 常断标志   | OFF | P_OFF | 该标志总是为 OFF（总是 0）。                                                                                                               |

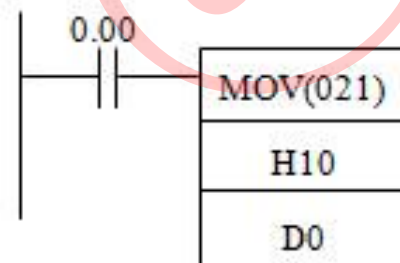


## 3.1 概述

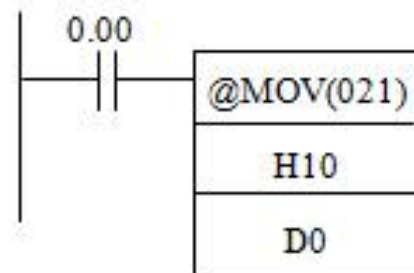
### 4. 应用指令的上升沿微分形式

指令类型：**周期性循环执行型**和**上升沿微分执行型**

- 周期性循环执行型：只要执行条件为ON，指令的周期性循环执行型在每个循环周期都将执行
- 上升沿微分执行型：仅在执行条件由OFF变为ON时才执行一次
- 绝大多数应用指令可在**周期性循环执行型**指令前加“@”符号作为**上升沿微分执行型**指令使用



(a) 周期性循环执行型



(b) 上升沿微分执行型

MOV 指令的两种形式



## 3.2 基本指令

### 3.2.1 顺序输入/顺序输出指令

#### 1. LD和LD NOT



|                                            |
|--------------------------------------------|
| N: 继电器编号                                   |
| CIO,W,H,A,TR,T,C,TK,<br>状态标志, 时钟标志, IR间接寻址 |

**功能：**表示常开（闭）触点与左侧母线连接；LD、LD NOT指令只能以位为单位进行操作，且不影响标志位。

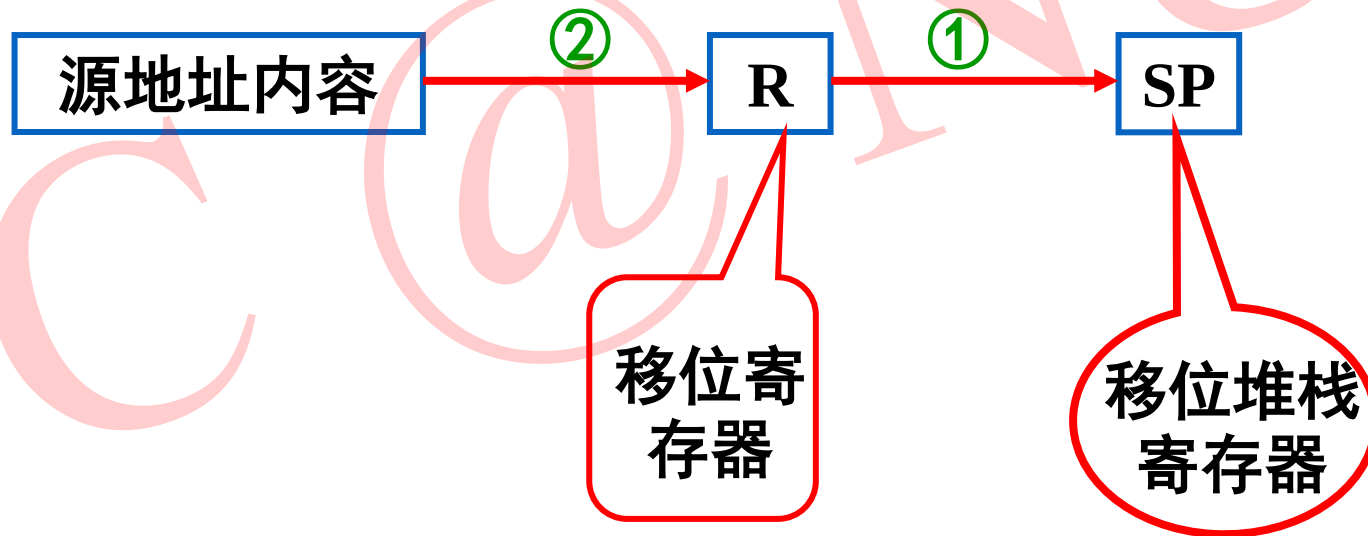


## 3.2 基本指令

### 3.2.1 顺序输入/顺序输出指令

#### 1. LD和LD NOT

LD指令操作流程圖：



➤ LD操作数所指定的源地址内容不变。

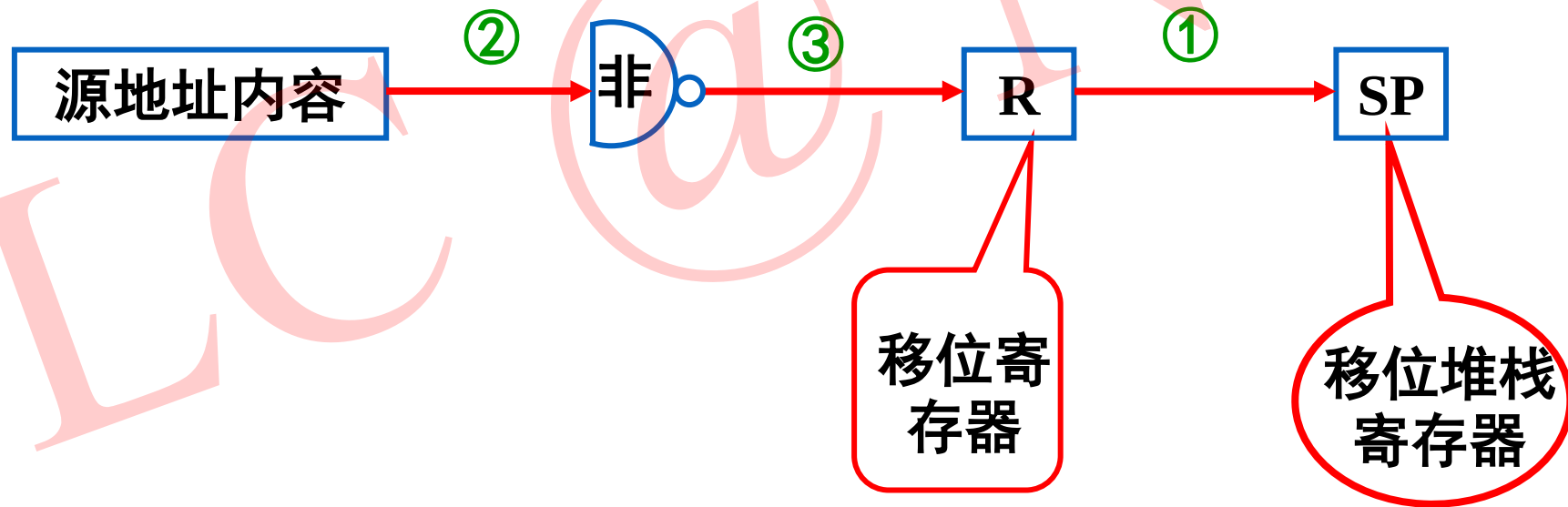


## 3.2 基本指令

### 3.2.1 顺序输入/顺序输出指令

#### 1. LD和LD NOT

LD NOT指令操作流程图：

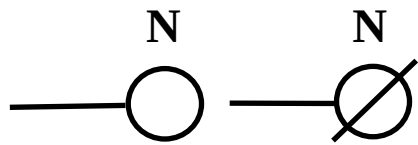




## 3.2 基本指令

### 3.2.1 顺序输入/顺序输出指令

#### 2. OUT和OUT NOT



|                     |
|---------------------|
| N: 继电器编号            |
| CIO,W,H,A,TR,IR间接寻址 |

**功能：**表示输出逻辑运算结果

**说明：**

- OUT、OUT NOT指令只能以位为单位进行操作，且不影响标志位
- CIO区中已用作输入通道的位不能作为OUT、OUT NOT的输出位
- OUT、OUT NOT常用于一条梯形图支路的最后，有时也用于分支点
- 线圈并联输出时，可连续使用OUT、OUT NOT指令

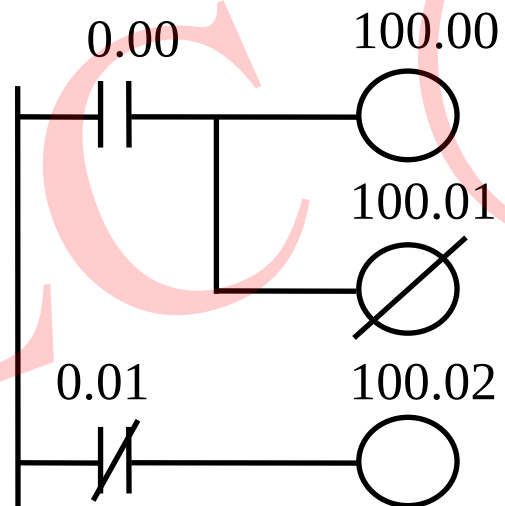


## 3.2 基本指令

### 3.2.1 顺序输入/顺序输出指令

#### 2. OUT和OUT NOT

➤ 线圈并联输出时，可连续使用OUT、OUT NOT指令。



|         |        |
|---------|--------|
| LD      | 0.00   |
| OUT     | 100.00 |
| OUT NOT | 100.01 |
| LD NOT  | 0.01   |
| OUT     | 100.02 |

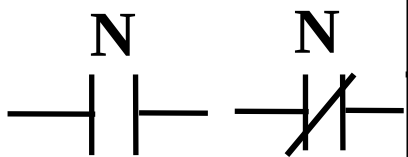




## 3.2 基本指令

### 3.2.1 顺序输入/顺序输出指令

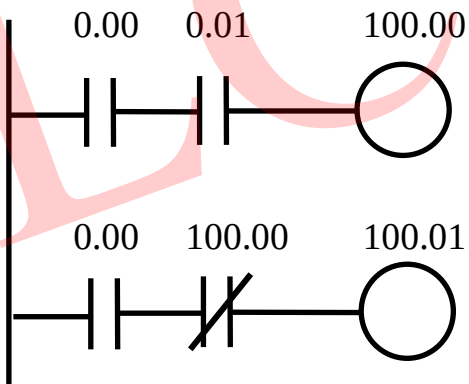
#### 3. AND和AND NOT指令



N: 继电器编号

CIO,W,H,A,TR,T,C,TK,状态标志,时钟标志,IR间接寻址

**功能：**表示常开触点与前面的触点电路相串联，后面的位与其前面的触点组逻辑“与”运算；AND NOT表示常闭触点与前面的触点电路相串联，或说取“反”后再与其前面的触点组进行逻辑“与”运算。



|         |        |
|---------|--------|
| LD      | 0.00   |
| AND     | 0.01   |
| OUT     | 100.00 |
| LD      | 0.01   |
| AND NOT | 100.00 |
| OUT     | 100.01 |

#### 说 明：

- 只能以位为单位进行操作，且不影响标志位
- 串联触点的个数没有限制

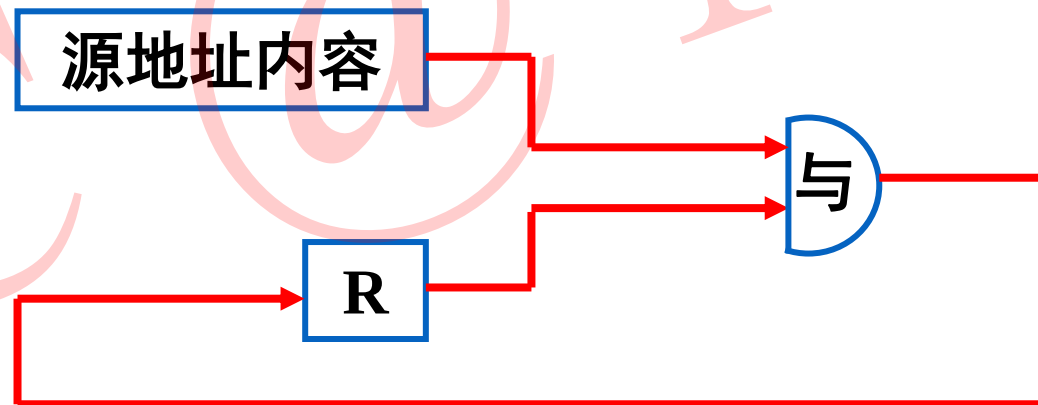


## 3.2 基本指令

### 3.2.1 顺序输入/顺序输出指令

#### 3. AND和AND NOT指令

AND指令操作流程图中：



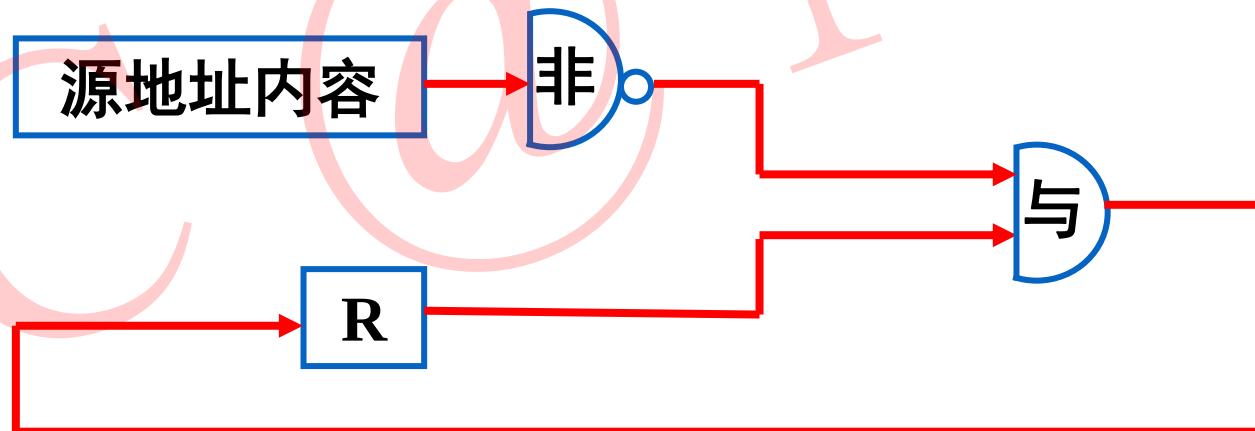


## 3.2 基本指令

### 3.2.1 顺序输入/顺序输出指令

#### 3. AND和AND NOT指令

AND NOT指令操作流程:



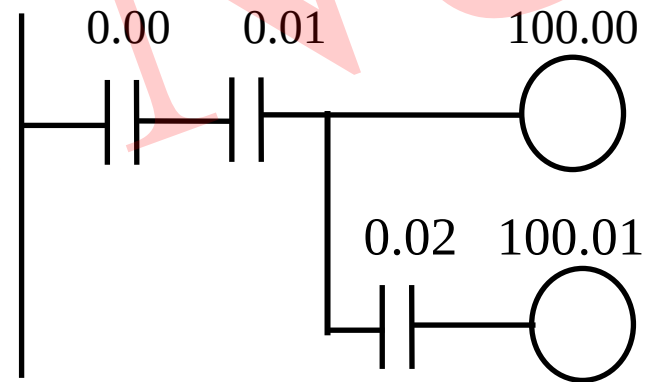
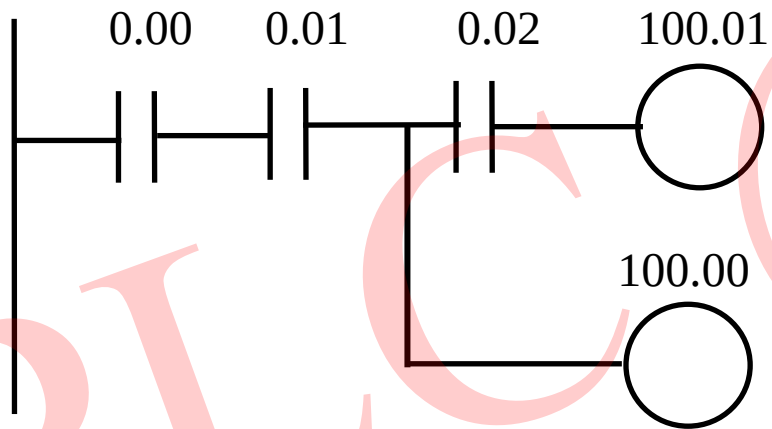


## 3.2 基本指令

### 3.2.1 顺序输入/顺序输出指令

### 3. AND和AND NOT指令

#### ◆ 连续输出次序颠倒



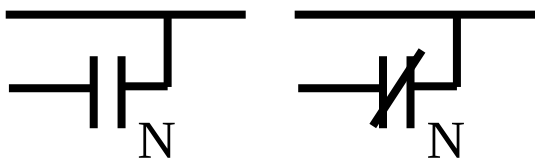
|     |        |
|-----|--------|
| LD  | 0.00   |
| AND | 0.01   |
| OUT | 100.00 |
| AND | 0.02   |
| OUT | 100.01 |



## 3.2 基本指令

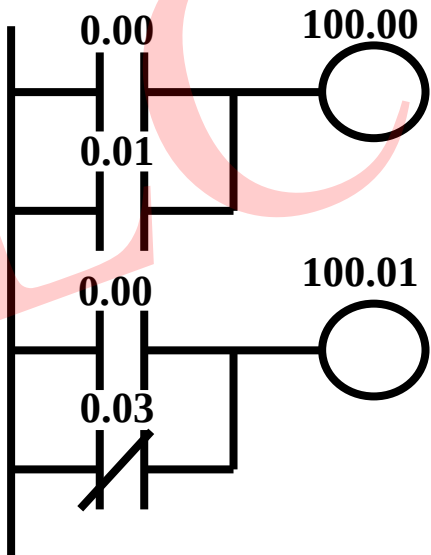
### 3.2.1 顺序输入/顺序输出指令

### 4. OR和OR NOT指令



|                                      |
|--------------------------------------|
| N: 继电器编号                             |
| CIO,W,H,A,TR,T,C,TK,状态标志,时钟标志,IR间接寻址 |

**功能：**表示常开触点与前面的触点电路相并联，后面的位与其前面的触点组逻辑“或”运算；OR NOT表示常闭触点与前面的触点电路相并联，或说取“反”后再与其前面的触点组进行逻辑“或”运算。



|        |        |
|--------|--------|
| LD     | 0.00   |
| OR     | 0.01   |
| OUT    | 100.00 |
| LD     | 0.02   |
| OR NOT | 0.03   |
| OUT    | 100.01 |

#### 说 明：

- ✓ 指令只能以位为单位进行操作，且不影响标志位
- ✓ 并联触点的个数没有限制



## 3.2 基本指令

### ■ 常用的基本指令小结

LD      LD NOT

AND    AND NOT

OR      OR NOT

OUT    OUT NOT

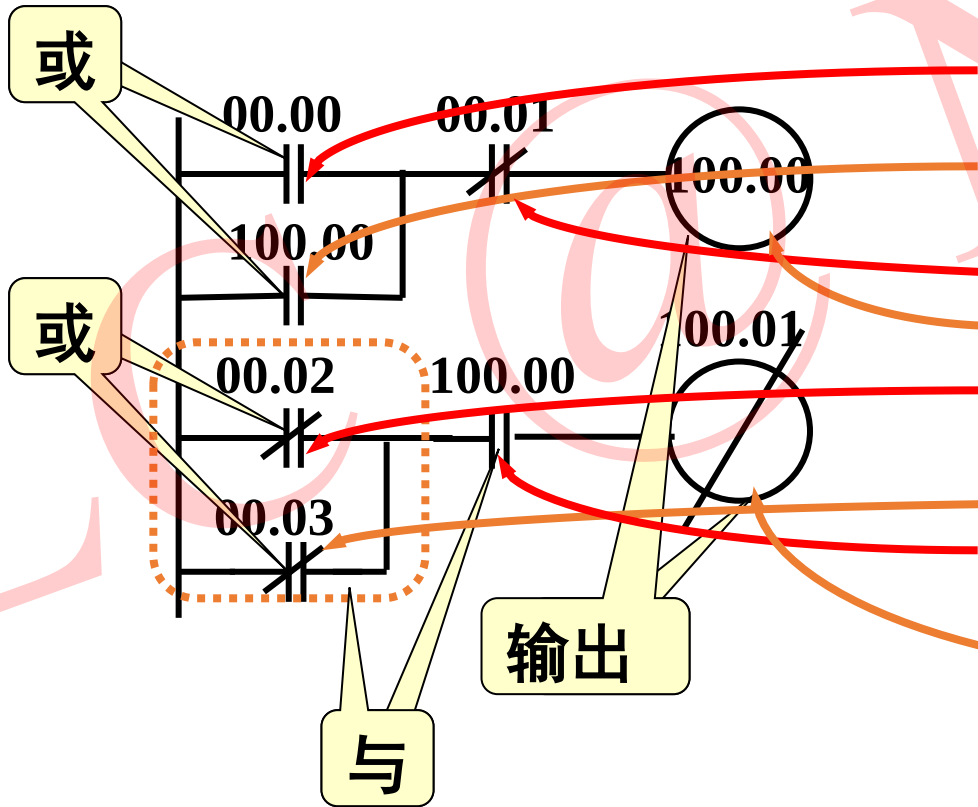
- LD（取指令）常开触点与母线相连的指令
- AND（与指令）常开触点相与（串联）的指令
- OR（或指令）常开触点相或（并联）的指令
- OUT（输出指令）将执行结果输出到某位的指令



# 3.2 基本指令

## 使用指令举例

### 梯形图程序



### 语句表程序

|         |        |
|---------|--------|
| LD      | 00.00  |
| OR      | 100.00 |
| AND NOT | 00.01  |
| OUT     | 100.00 |
| LD NOT  | 00.02  |
| OR NOT  | 00.03  |
| AND     | 100.00 |
| OUT NOT | 100.01 |



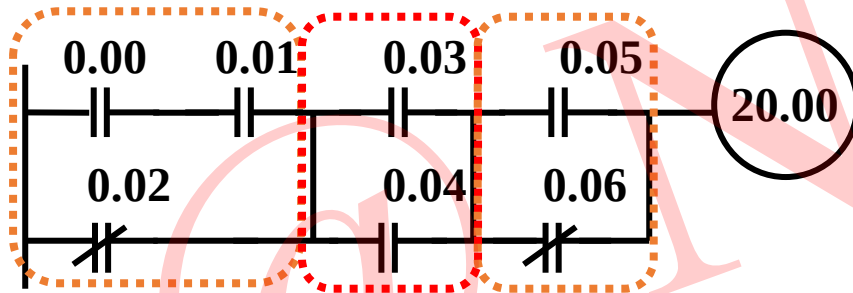
## 3.2 基本指令

### 5. AND LD指令

**功能：**用于逻辑块串联连接，即对逻辑块进行逻辑“与”的操作。每个逻辑块都以LD或LD NOT指令开始。AND LD指令单独使用，**无操作数**。

方法1

```
LD      0.00
AND     0.01
OR NOT  0.02
LD      0.03
OR      0.04
AND LD
LD      0.05
OR NOT  0.06
AND LD
OUT     20.00
```



✓ **AND LD是将并联触点组相串联的指令**

方法2

```
LD      0.00
AND     0.01
OR NOT  0.02
LD      0.03
OR      0.04
LD      0.05
OR NOT  0.06
AND LD
AND LD
OUT     20.00
```

在方法2中，AND LD指令之前的逻辑块数应小于等于8，而方法1对此没有限制





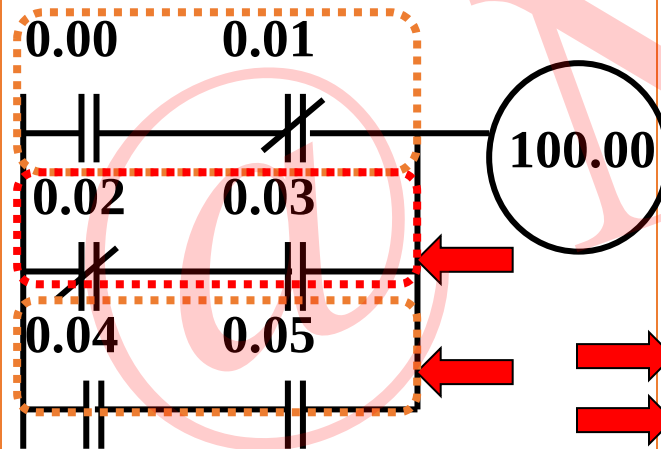
## 3.2 基本指令

### 6. OR LD

**功能：**用于逻辑块并联连接，即对逻辑块进行逻辑“或”的操作。每个逻辑块都以LD或LD NOT指令开始。OR LD指令单独使用，无操作数。

#### 方法1

|         |        |
|---------|--------|
| LD      | 0.00   |
| AND NOT | 0.01   |
| LD NOT  | 0.02   |
| AND     | 0.03   |
| OR LD   |        |
| LD      | 0.04   |
| AND     | 0.05   |
| OR LD   |        |
| OUT     | 100.00 |



#### 方法2

|         |        |
|---------|--------|
| LD      | 0.00   |
| AND NOT | 0.01   |
| LD NOT  | 0.02   |
| AND     | 0.03   |
| LD      | 0.04   |
| AND     | 0.05   |
| OR LD   |        |
| OR LD   |        |
| OUT     | 100.00 |

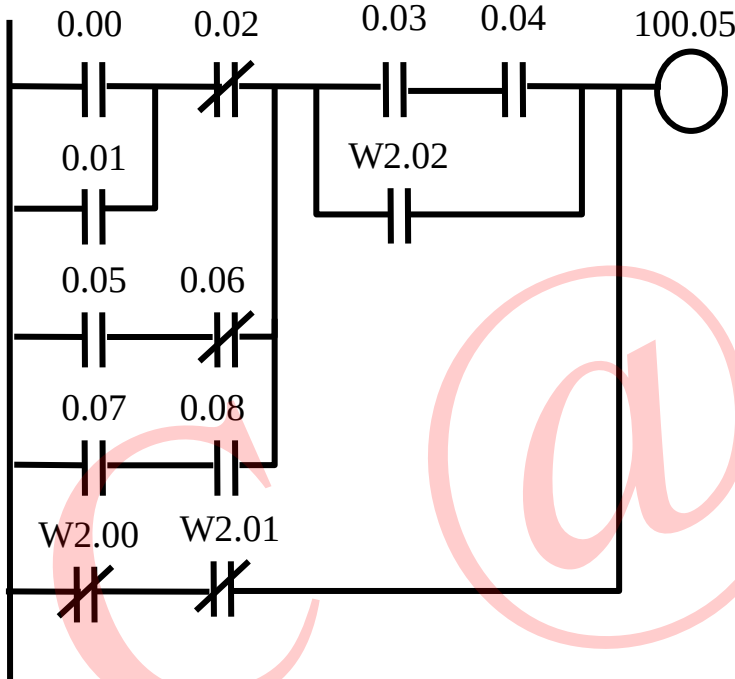
✓ OR LD是串联触点组相并联连接的指令

在方法2中，OR LD指令之前的逻辑块数应小于等于8，而方法1对此没有限制



# 3.2 基本指令

## ◆ 块与、块或指令的应用



复杂梯形图

|         |        |
|---------|--------|
| LD      | 0.00   |
| OR      | 0.01   |
| AND NOT | 0.02   |
| LD      | 0.05   |
| AND NOT | 0.06   |
| OR LD   |        |
| LD      | 0.07   |
| AND     | 0.08   |
| OR LD   |        |
| LD      | 0.03   |
| AND     | 0.04   |
| OR      | W2.02  |
| AND LD  |        |
| LD NOT  | W2.00  |
| AND NOT | W2.01  |
| OR LD   |        |
| OUT     | 100.05 |

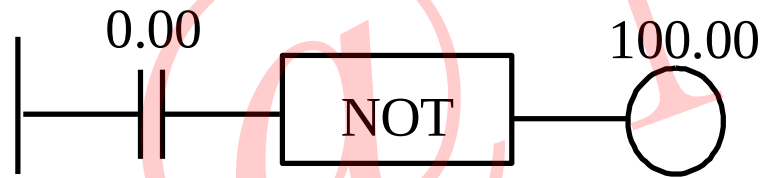


## 3.2 基本指令

### 7. 非指令NOT



**功能：**将输入条件取反，连接到下一段

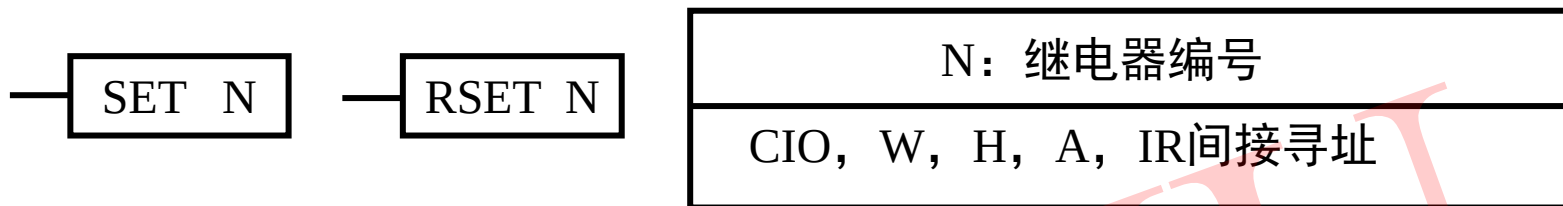


当0.00为OFF时，经过NOT取反，100.00为ON；当0.00为ON时，经过NOT取反，100.00为OFF。



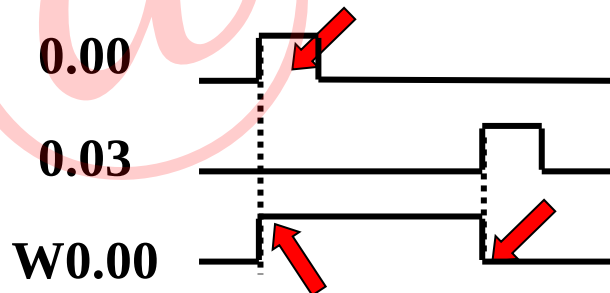
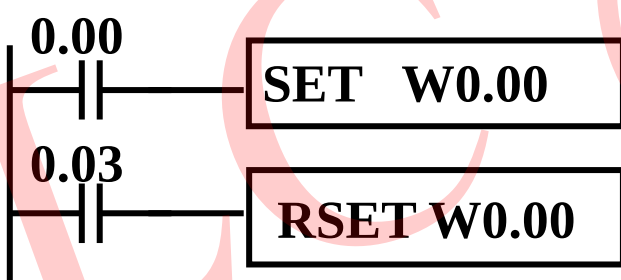
## 3.2 基本指令

### 8. 置位和复位指令—SET和RSET



SET、RSET指令的梯形图符号及操作数取值区域

**功能：**执行条件为ON时，SET使指定继电器置位为ON；  
执行条件为ON时，RSET使指定继电器复位为OFF。



|      |       |
|------|-------|
| LD   | 0.00  |
| SET  | W0.00 |
| LD   | 0.03  |
| RSET | W0.00 |

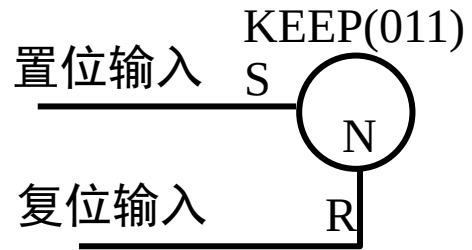
SET、RSET指令的执行条件常为短信号（脉冲信号）

SET和RSET指令可以单独使用（不一定成对使用）



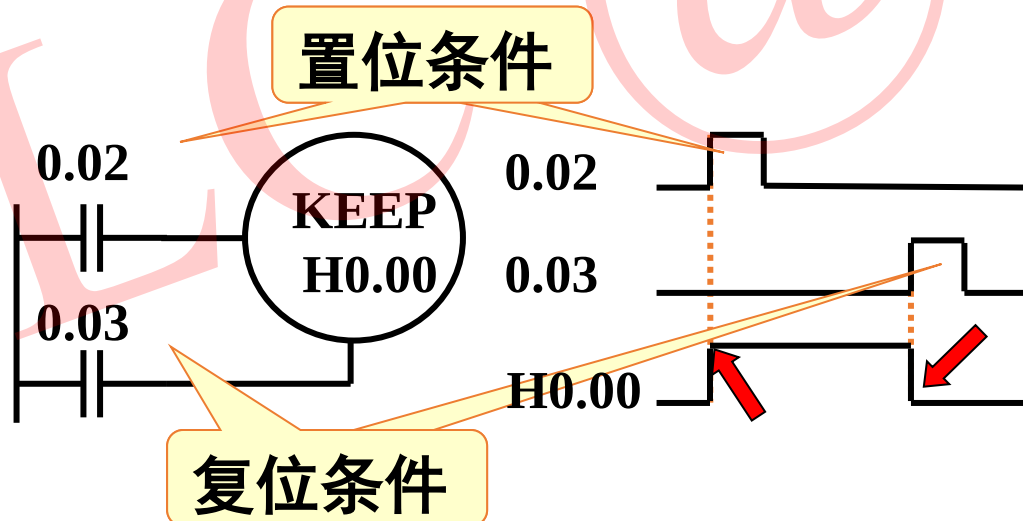
## 3.2 基本指令

### 9. 保持指令—KEEP(011) 锁存指令



|                      |
|----------------------|
| N: 继电器编号             |
| CIO, W, H, A, IR间接寻址 |

**功能：**当S端为ON时，继电器N保持为ON状态直至R端为ON时使其变为OFF。当两个输入端同时为ON时继电器N为OFF。



注意！

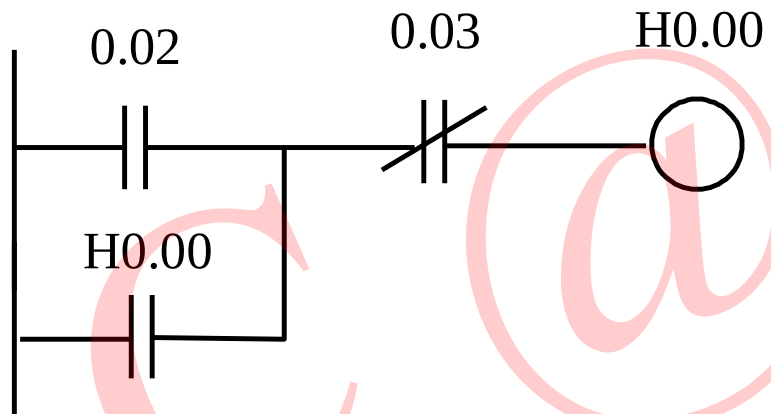
LD 0.02  
LD 0.03  
KEEP (11) H0.00



## 3.2 基本指令

### 9. 保持指令—KEEP(011)

#### ◆ 使用KEEP指令的启保停电路



启保停电路

|         |       |
|---------|-------|
| LD      | 0.02  |
| OR      | H0.00 |
| AND NOT | 0.03  |
| OUT     | H0.00 |

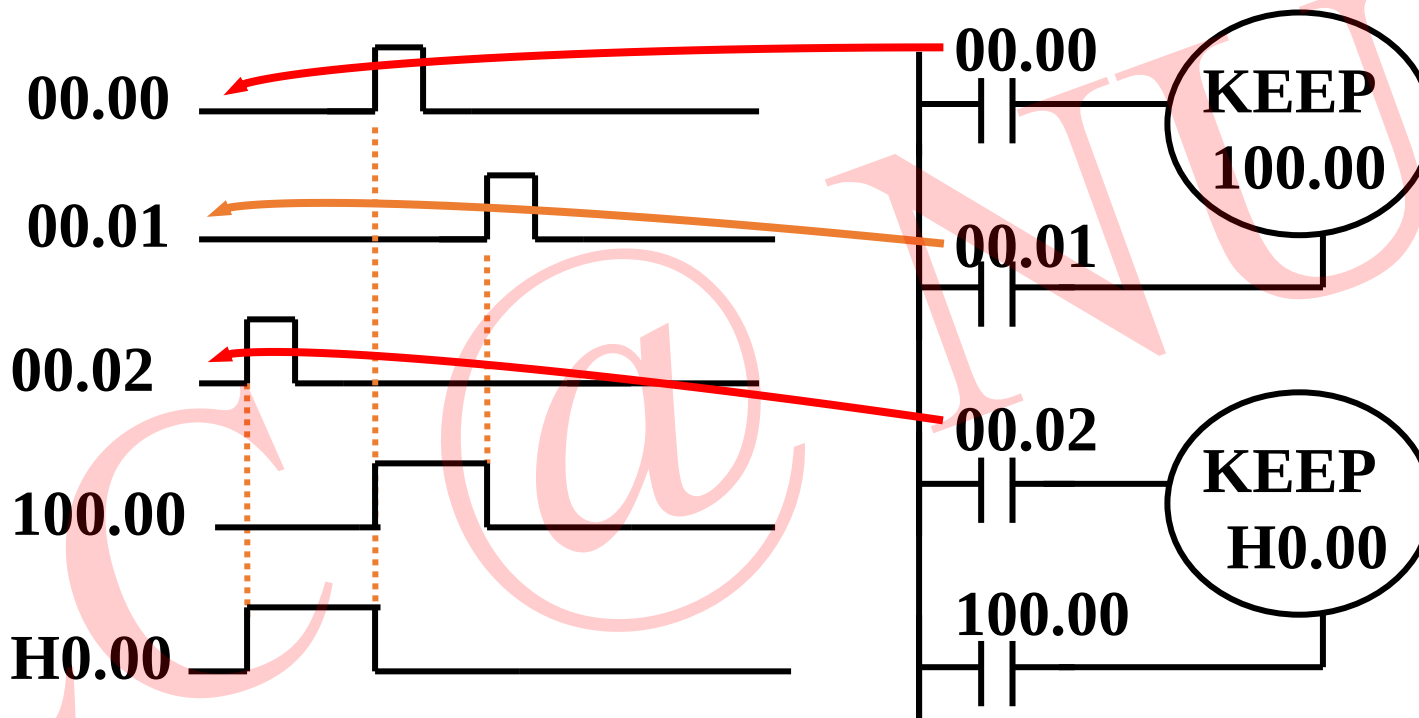


## 3.2 基本指令

### 9. 保持指令—KEEP(011)

✓ 使用KEEP指令举例

画出图示程序的工作时序，写出语句表。



语  
句  
表

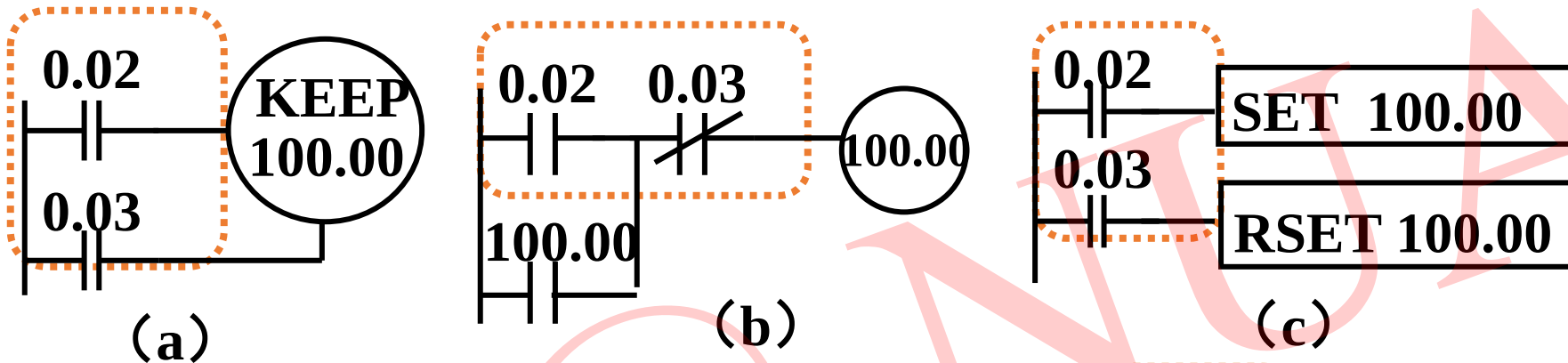
|      |        |
|------|--------|
| LD   | 00.00  |
| LD   | 00.01  |
| KEEP | 100.00 |

|      |        |
|------|--------|
| LD   | 00.02  |
| LD   | 100.00 |
| KEEP | H0.00  |



## 3.2 基本指令

### ★ 比较几个相同具有功能程序之区别



**相同** 都可以实现启、保、停的控制

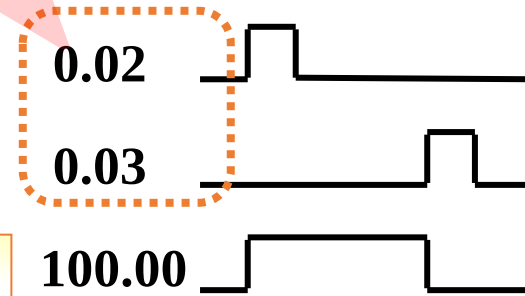
KEEP编程需3条语句（最少）

KEEP编程用HR作输出时，具有保持功能。

**不同**

SET和RSET编程HR作输出时，有保持功能。

SET和RSET编程时，指令间可插别的指令。

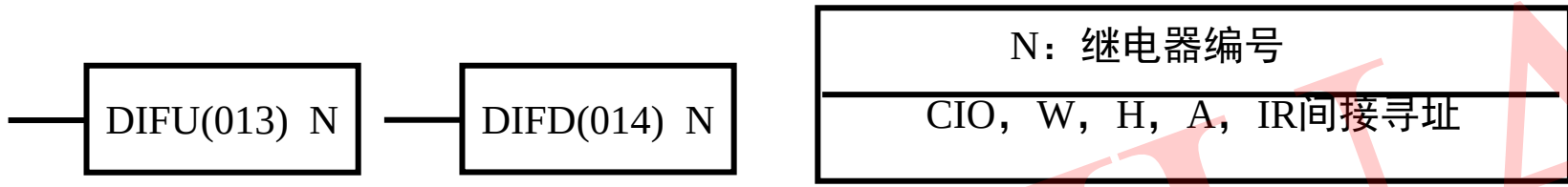






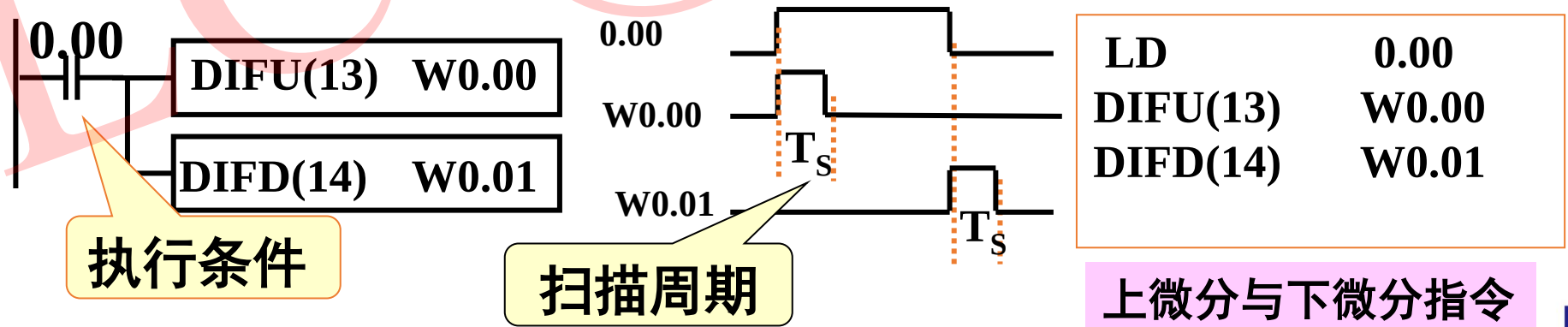
### 3.2 基本指令

#### 10. 上升沿微分DIFU (013) 和下降沿微分指令DIFD (014)



DIFU、DIFD指令的梯形图符号及操作数取值区域

**功能：**当执行条件由OFF变为ON时，上升沿微分DIFU使指定继电器在一个扫描周期内为ON；当执行条件由ON变为OFF时，下降沿微分指令DIFD使指定继电器在一个扫描周期内为ON。





## 3.2 基本指令

### 10. 上升沿微分DIFU (013) 和下降沿微分指令DIFD (014)

- CIO区中已用作输入通道的位不能作为微分指令的输出位，输出通道的位最好也不要
- 在第 $n$ 次扫描时检测到输入条件为OFF，第 $n+1$ 次扫描检测到ON时，DIFU指令才开始执行，同样在第 $n$ 次扫描时检测到输入条件为ON，第 $n+1$ 次扫描检测到OFF时，DIFD指令才开始执行

**DIFU和DIFD指令的操作位只ON一个扫描周期。**

**开机时就ON的执行条件，DIFU指令不执行。**

**开机时就OFF的执行条件，DIFD指令不执行。**

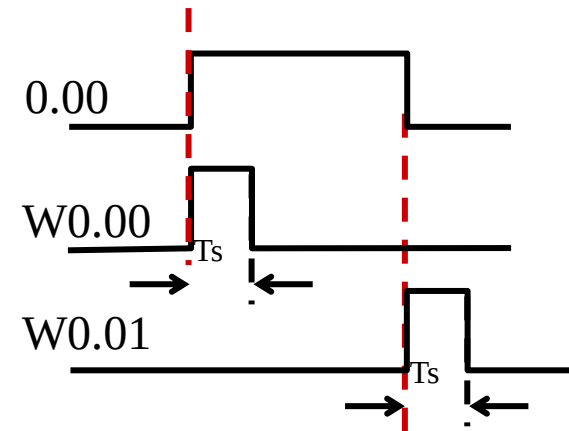
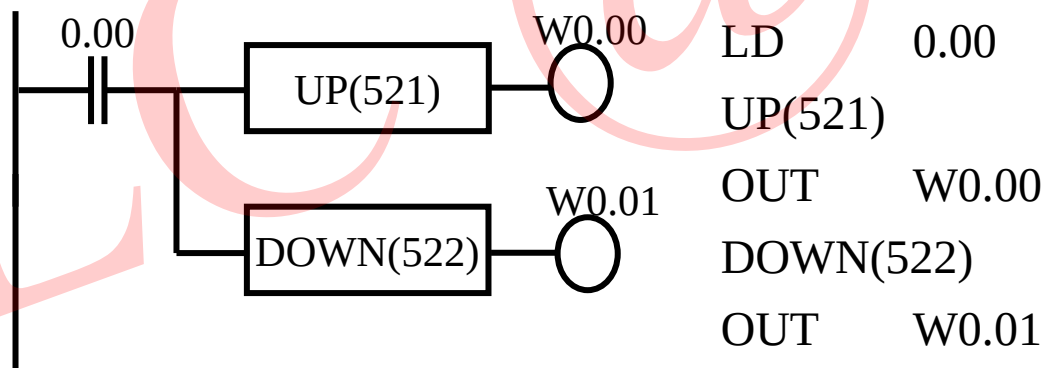


## 3.2 基本指令

### 11. 条件上升沿微分UP (521) 和条件下降沿微分DOWN (522)



**功能：**当执行条件由OFF变为ON时，条件上升沿微分UP输出ON一个扫描周期，连接到下一段；当执行条件由ON变为OFF时，条件下降沿微分DOWN输出ON一个扫描周期，连接到下一段。



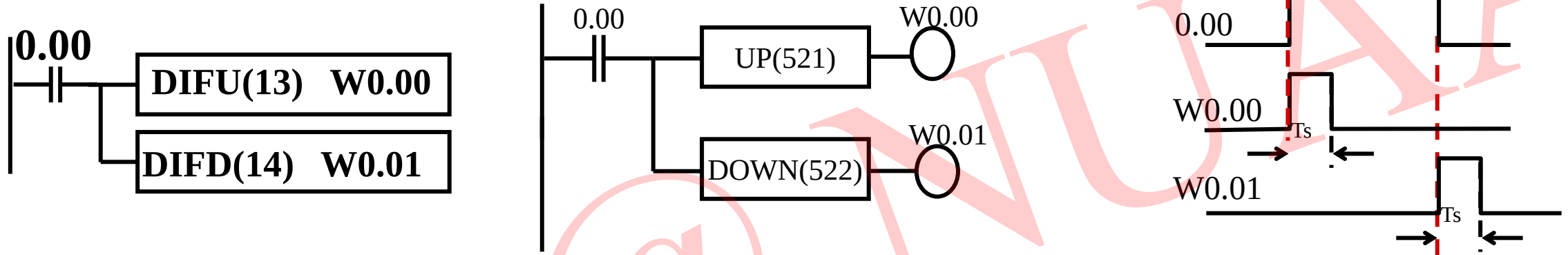
UP和DOWN指令的应用



## 3.2 基本指令

### 11. 条件上升沿微分UP (521) 和条件下降沿微分DOWN (522)

UP和DOWN指令的应用



这两段程序执行的结果是相同。

**区别：** 条件上升沿微分执行用在比如前面有好几个常开常闭串并联等等统一判断后的上升沿，上升沿微分指令是针对某个节点的上升沿。



## 3.2 基本指令

### 3.2.2 指令的微分执行和立即刷新执行

- LD/LD NOT、AND/AND NOT、OR/OR NOT和OUT/OUT NOT属于使用频率最高的基本指令
- 从指令执行的方式来看，LD/LD NOT、AND/AND NOT和OR/OR NOT除通常的周期性循环执行外，还有微分执行、立即刷新执行以及二者组合执行；OUT/OUT NOT除通常的周期性循环执行外，还有立即刷新执行
- 微分有上微分和下微分，上微分在语句表中用“@”标识，在梯形图中以“↑”标识；下微分在语句表中用“%”标识，在梯形图中以“↓”标识。立即刷新无论在语句表还是梯形图中均以“!”标识



# 常用指令的微分执行与立即刷新执行

| 执行方式      | 指 令                                                            | 功 能                                      | I/O刷新 |
|-----------|----------------------------------------------------------------|------------------------------------------|-------|
| 周期性循环执行   | <b>LD, AND, OR<br/>LD NOT, AND NOT, OR NOT</b>                 | 指定触点的状态在周期性I/O刷新时由CPU读入, 进行相应的运算, 产生执行结果 | 周期性刷新 |
|           | <b>OUT, OUT NOT</b>                                            | 指令执行后, 将指定线圈的状态在之后的周期性I/O刷新阶段输出          |       |
| 上微分执行     | <b>@LD, @AND, @OR<br/>@LD NOT, @AND NOT,<br/>@OR NOT</b>       | 在指定触点上升沿时执行指令, 且仅执行一个周期                  |       |
| 下微分执行     | <b>%LD, %AND, %OR<br/>%LD NOT, %AND NOT,<br/>%OR NOT</b>       | 在指定触点下降沿时执行指令, 且仅执行一个周期                  |       |
| 立即刷新执行    | <b>!LD, !AND, !OR<br/>!LD NOT, !AND NOT, !OR NOT</b>           | 立即读入指定触点的ON/OFF状态, 并执行指令                 | 指令执行前 |
|           | <b>!OUT, !OUT NOT</b>                                          | 执行指令的结果并立即输出给指定线圈                        | 指令执行后 |
| 上微分立即刷新执行 | <b>!@LD, !@AND, !@OR<br/>!@LD NOT, !@AND NOT, !@OR NOT</b>     | 立即读入指定触点的状态, 在其上升沿执行指令, 且仅执行一个周期         | 指令执行前 |
| 下微分立即刷新执行 | <b>!%LD, !%AND, !%OR<br/>!%LD NOT, !%AND NOT,<br/>!%OR NOT</b> | 立即读入指定触点的状态, 在其下降沿执行指令, 且仅执行一个周期         |       |



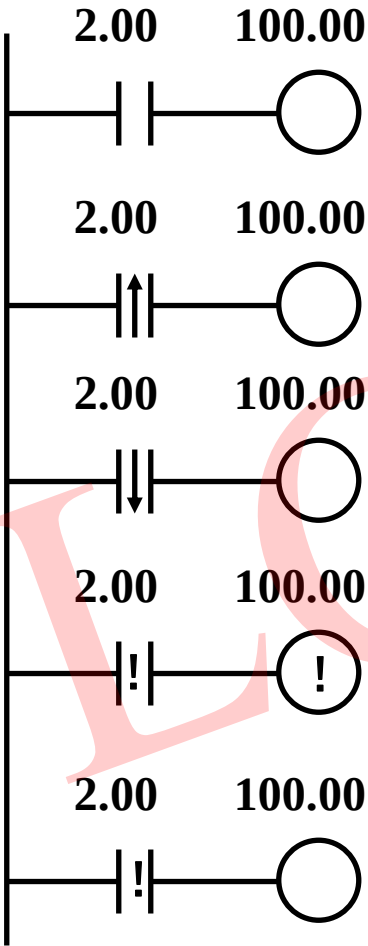
# 常用指令的微分执行与立即刷新执行

## 说明：

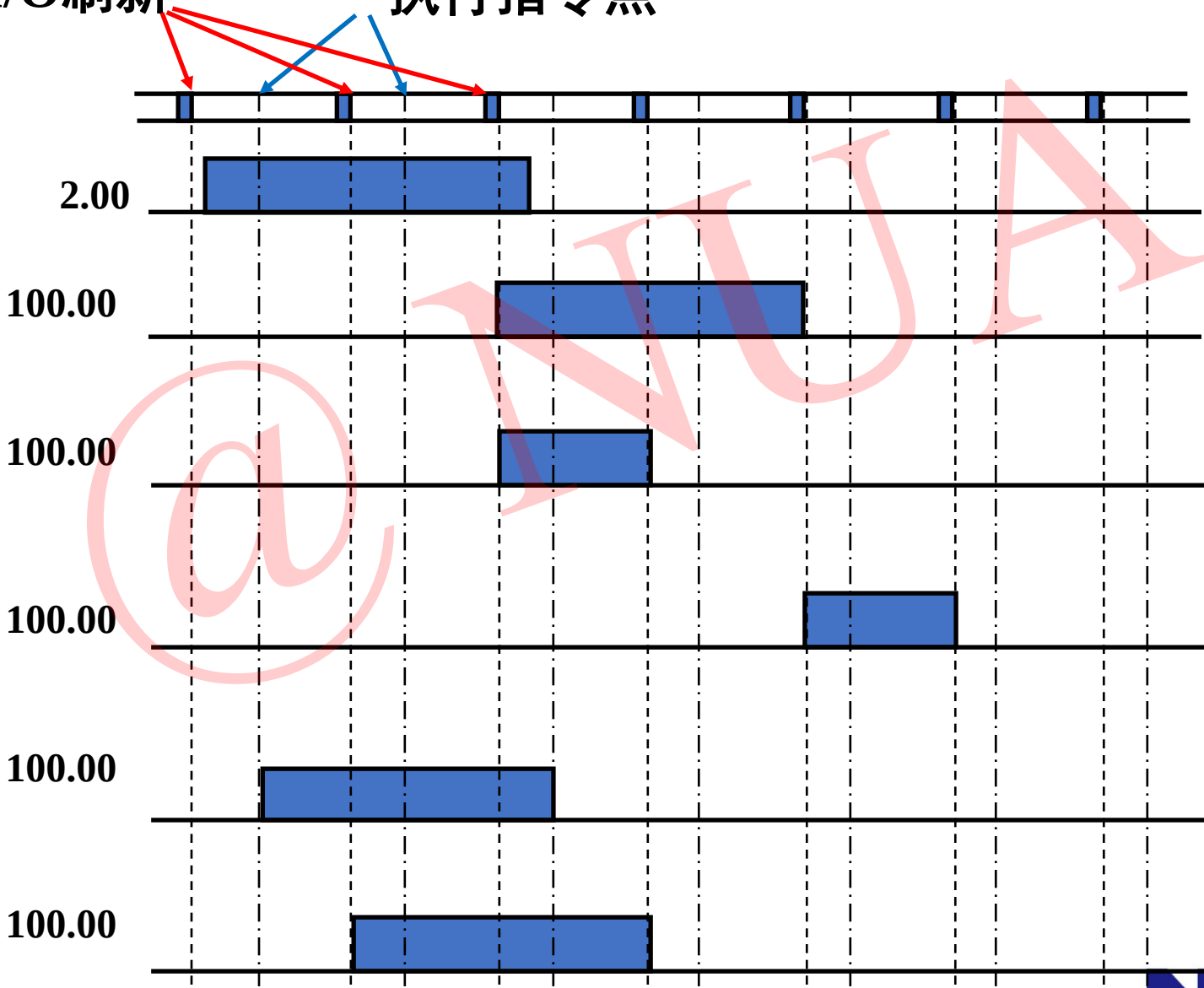
- ❑ 指令的立即刷新执行仅适用于CPU单元内置的I/O点，不能用于扩展的I/O点，后者可使用IORF指令实现立即刷新功能。
- ❑ CP1 PLC指令除周期性循环执行外，大多数可上微分执行，但可下微分执行的指令为少数，除表中所列，仅还有%SET、%RSET。
- ❑ CP1 PLC指令中，可立即刷新执行的也为少数，除表3.4中所列，还有!SET、!RSET、!KEEP、!  
DIFU、!DIFD、!CMP、!MOV、!CPS、!SETB、!RSTB、!OUTB。



由LD指令和OUT指令构成的程序，执行的时序如图



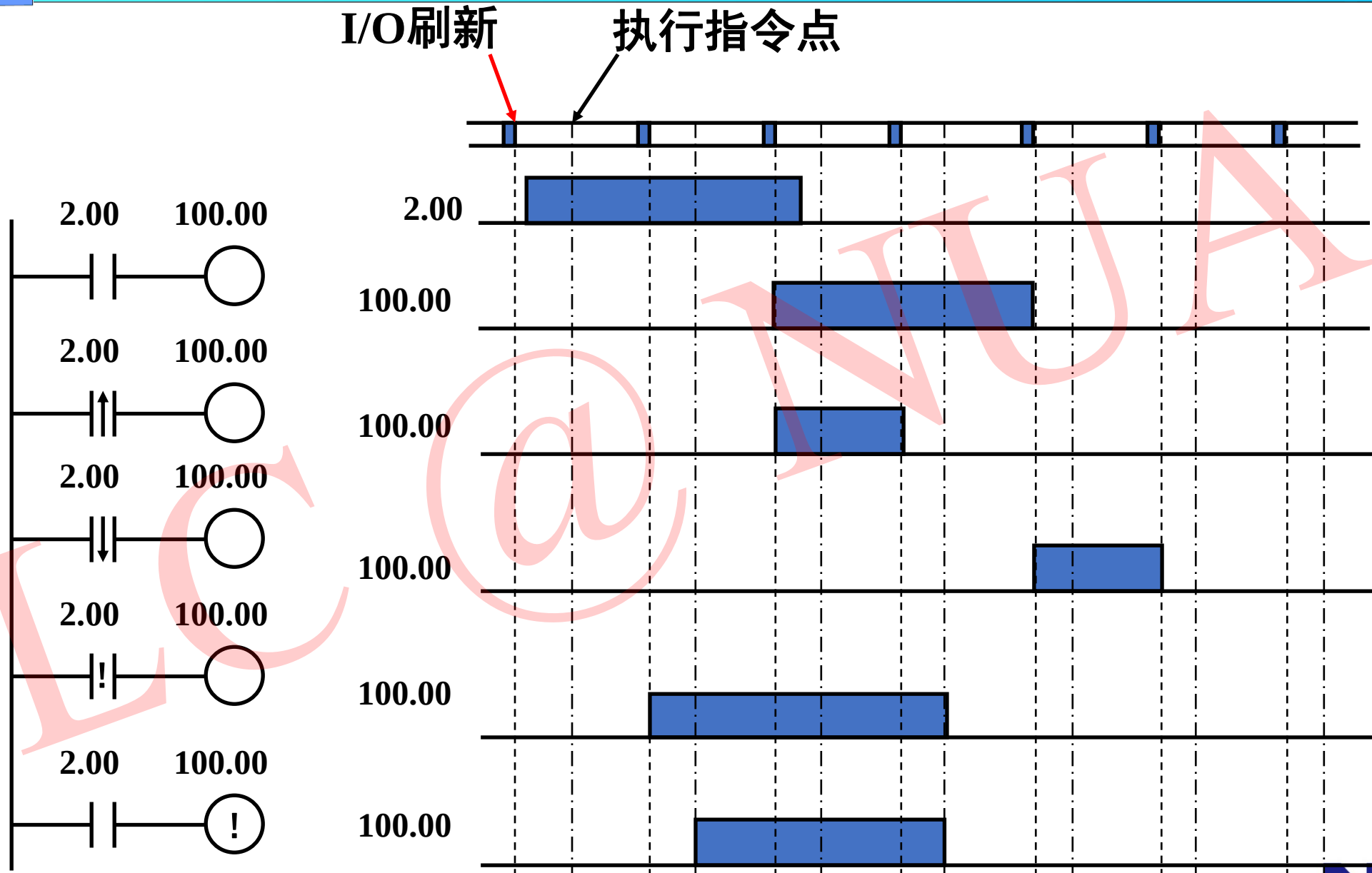
I/O刷新      执行指令点





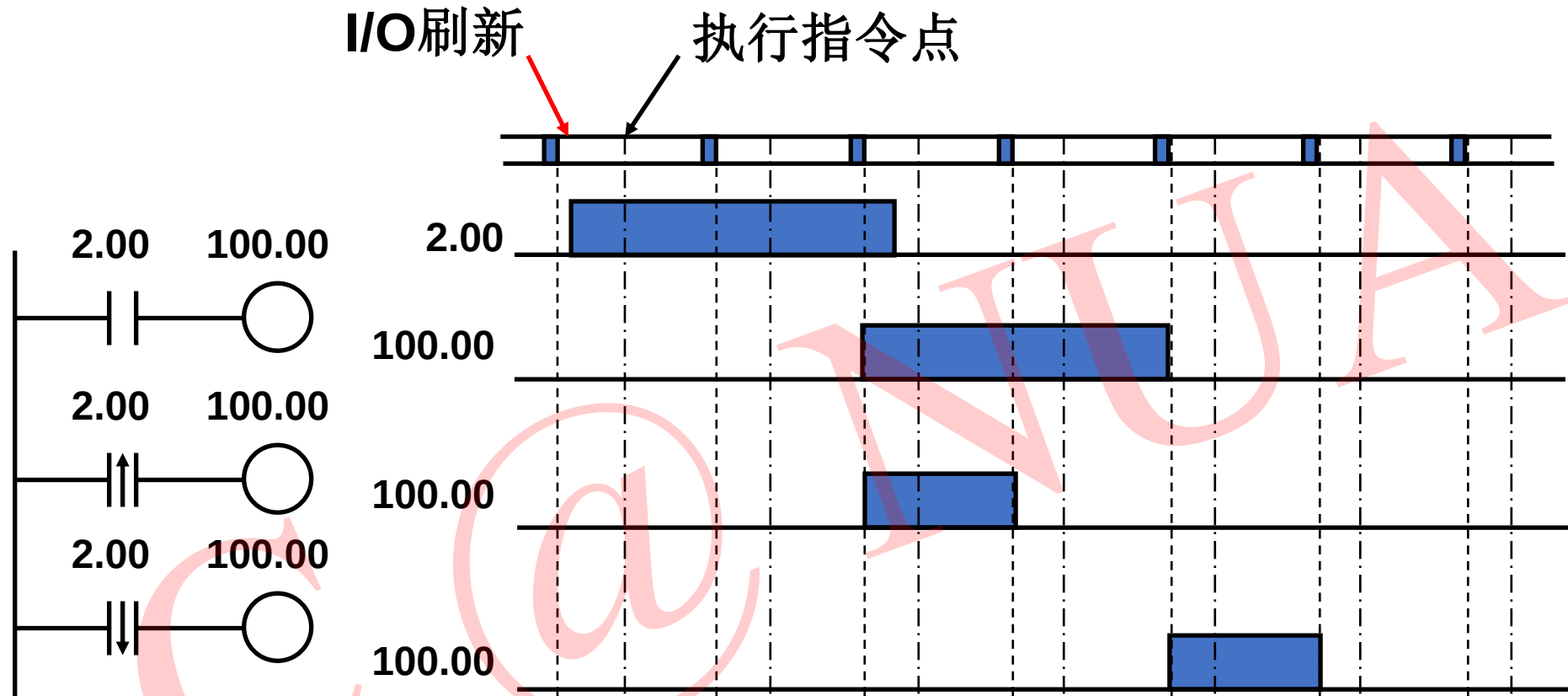


# 由LD指令和OUT指令构成的程序，执行的时序如图





## 由LD指令和OUT指令构成的程序，执行的时序如图

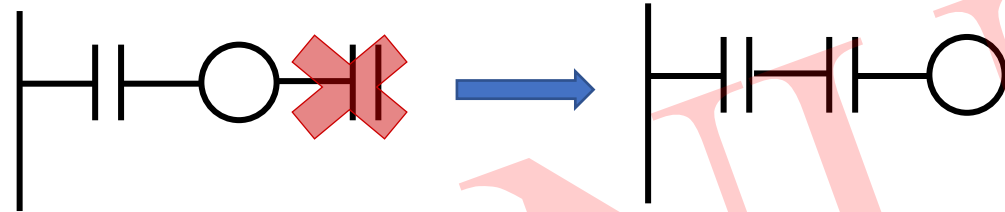




## 3.2 基本指令

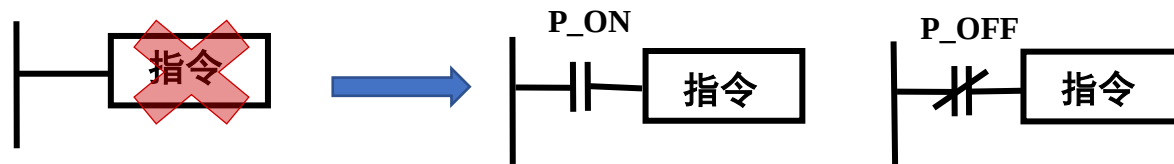
### 3.2.3 编程规则及技巧

1. 梯形图中线圈应放在最右边



2. 除极少数指令(如ILC、JME等)不允许有执行条件外，几乎所有的指令都需要执行条件

◆ 如果一条指令在PLC上电后需要一直执行，不能直接连到母线上，可以将状态标志区的常ON标志（P\_ON）或常OFF标志（P\_OFF）取“反”后作为执行条件。



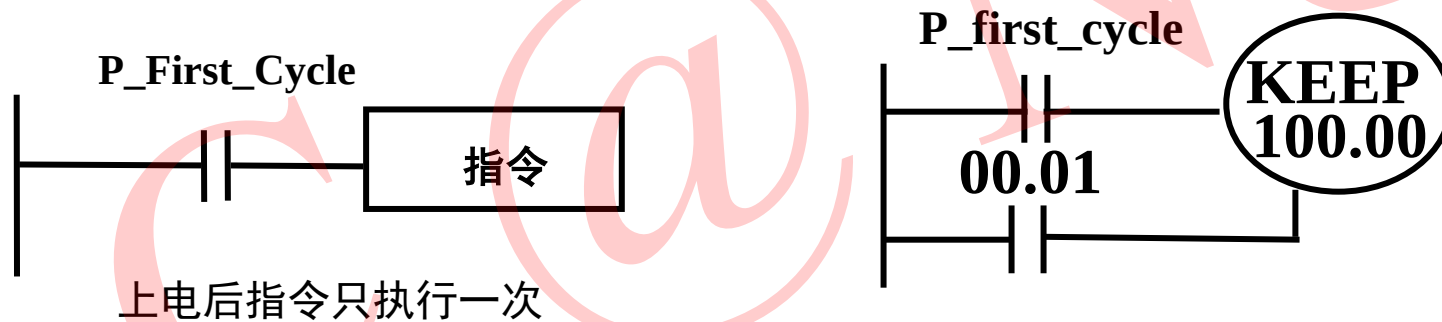
上电后指令一直执行



## 3.2 基本指令

### 3.2.3 编程规则及技巧

- ◆ 如果一条指令在PLC上电后只需要执行一次，可以将标志位P\_First\_Cycle作为执行条件。如图，P\_First\_Cycle在PLC运行的第一个扫描周期处于ON状态，然后处于OFF状态。这种用法常出现在PLC进行工作初始化的程序段上。



PLC上电后的第一个扫描周期，100.00被置为ON。此后，如果00.01 ON使100.00复位，则在PLC本次上电期间，100.00不会再被置位（即KEEP不再执行）。

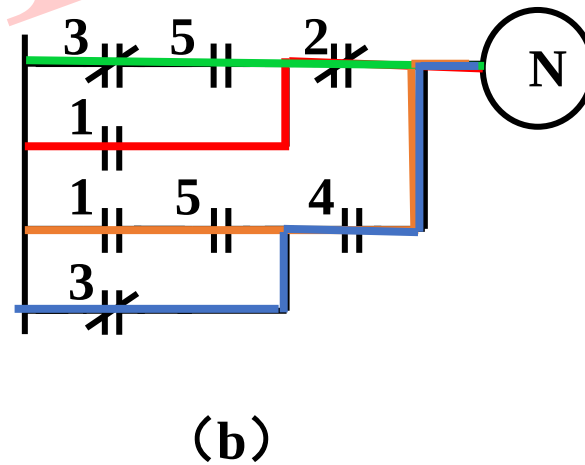
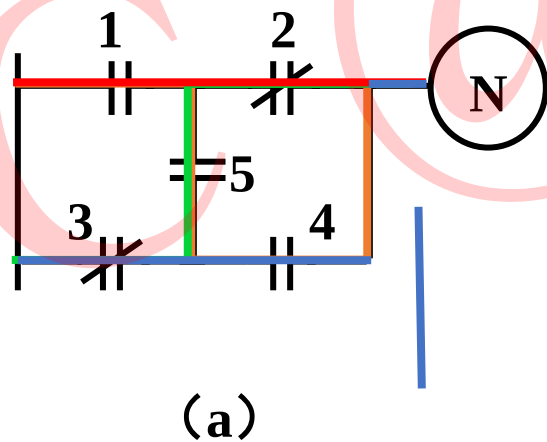


## 3.2 基本指令

### 3.2.3 编程规则及技巧

#### 3. 触点不能画在垂直路径上

❖ 梯形图必须遵循从左到右、从上到下的顺序，不允许两行之间垂直连接触点。



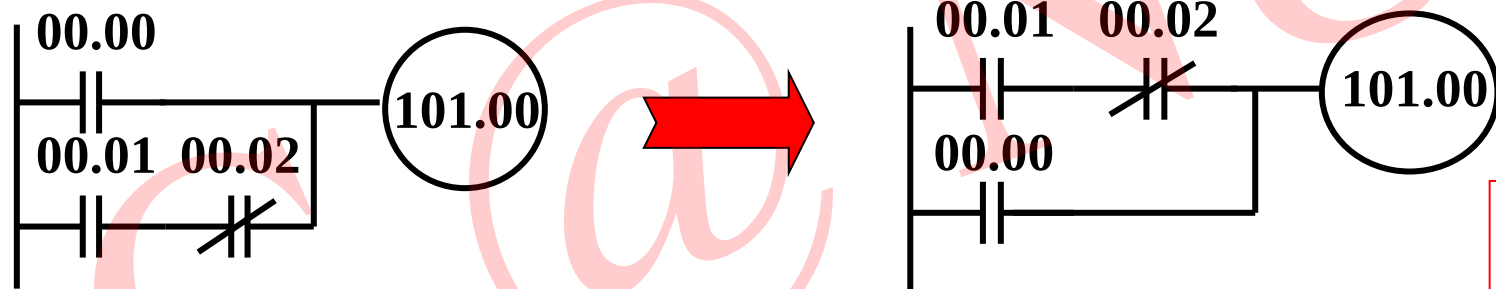


## 3.2 基本指令

### 3.2.3 编程规则及技巧

4. 编程时，对于逻辑关系复杂程序段，应按照先复杂后简单原则编程

- 触点组并联时，应将触点最多的那个串联电路放在梯形图最上面。



```
LD    00.00
LD    00.01
AND NOT 00.02
OR LD
OUT    101.00
```

```
LD    00.01
AND NOT 00.02
OR    00.00
OUT    101.00
```

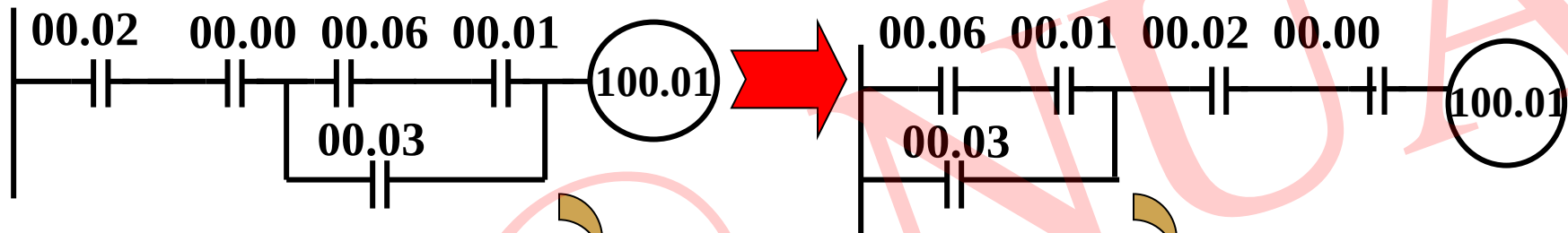
不必用  
OR LD



## 3.2 基本指令

### 3.2.3 编程规则及技巧

- 触点组串联时，应将触点最多的并联电路放在梯形图的最左边。



|        |       |
|--------|-------|
| LD     | 00.02 |
| AND    | 00.00 |
| LD     | 00.06 |
| AND    | 00.01 |
| OR     | 00.03 |
| AND LD |       |
| OUT    | 10.01 |

|     |        |
|-----|--------|
| LD  | 00.06  |
| AND | 00.01  |
| OR  | 00.03  |
| AND | 00.02  |
| AND | 00.00  |
| OUT | 100.01 |

不必用  
AND LD



## 3.2 基本指令

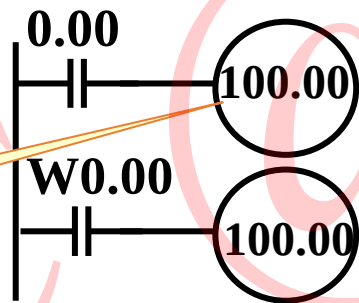
### 3.2.3 编程规则及技巧

#### (5) 避免出现双线圈输出

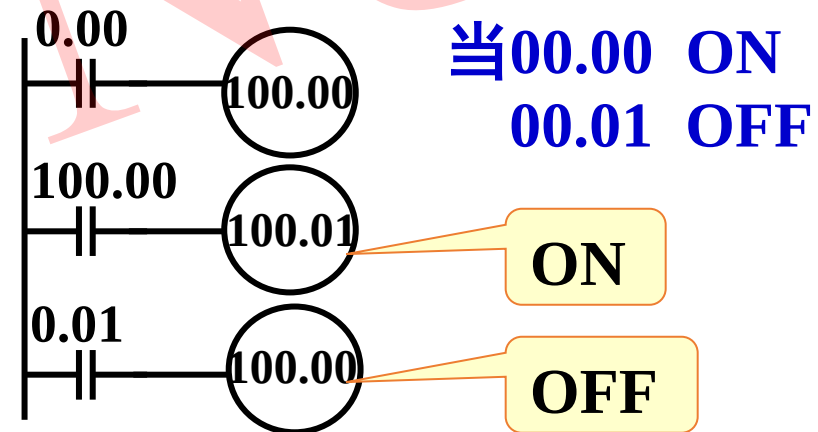
- 如果在同一个程序中，同一元件的线圈使用了两次或多次，称为双线圈输出。这时前面的输出无效，最后一次输出才是有效的。

当00.00 ON  
W0.00 OFF

输出无效



(a)



(b)

双线圈输出引起逻辑混乱

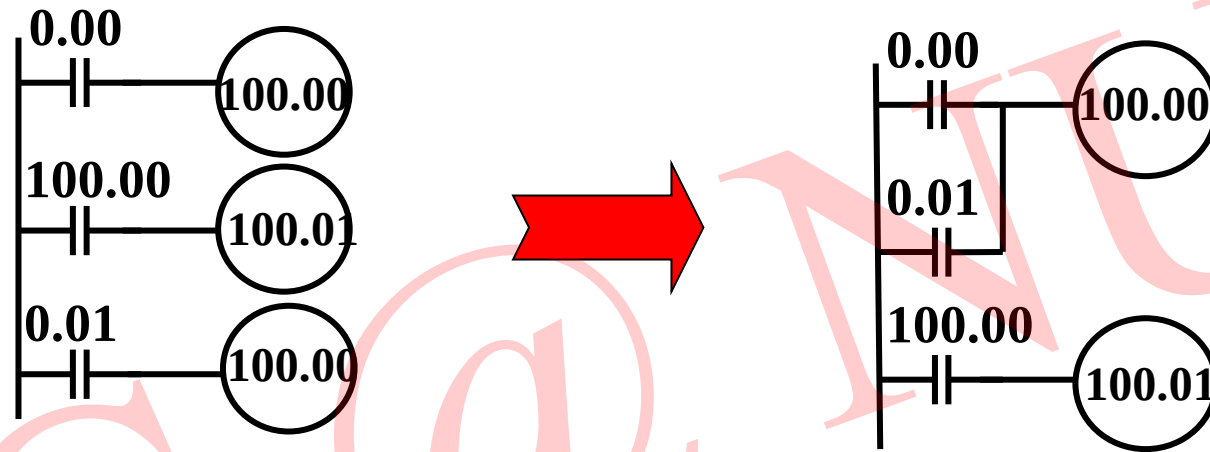




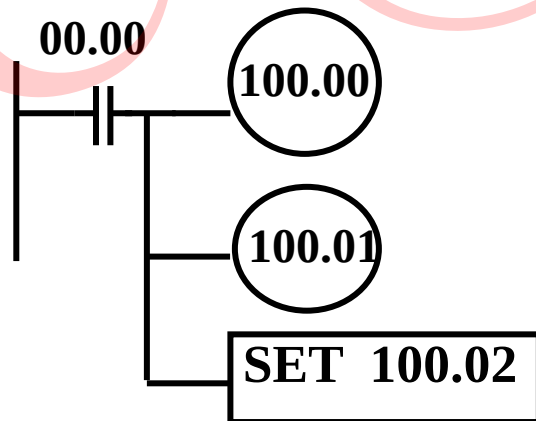
## 3.2 基本指令

### 3.2.3 编程规则及技巧

#### 5. 避免出现双线圈输出



➤ 两个或两个以上的线圈或指令可以并联输出。



|     |        |
|-----|--------|
| LD  | 00.00  |
| OUT | 100.00 |
| OUT | 100.01 |
| SET | 100.02 |



## 3.2 基本指令

### 3.2.3 编程规则及技巧

6. 程序结束一定要安排END指令，否则程序不被执行。没END指令的程序，检查时显示信息“NO END INST”



END 指令的梯形图符号

**功能：END指令表示程序段结束。END指令无操作数。**

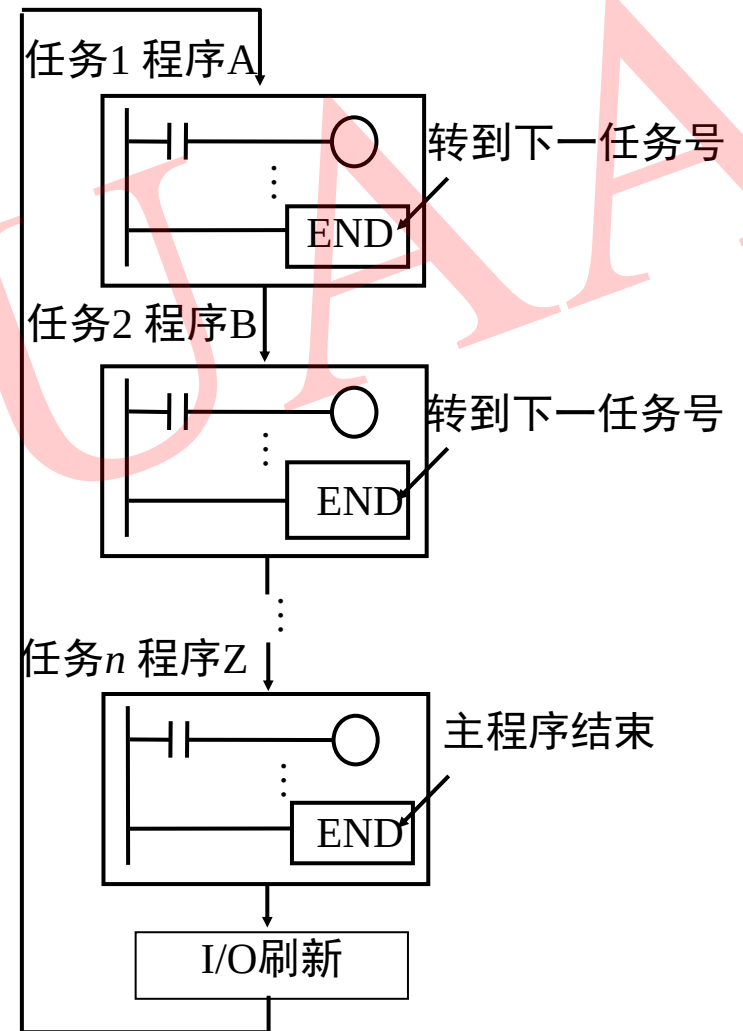
**说明：执行END指令时，ER、EQ、NE、CY、GT、LT等标志位都被置为OFF。**



## 3.3 顺序控制指令

### 1. 结束指令—END (001)

- CP1 PLC的程序划分为多个任务，每个任务对应一个程序段，采用多任务顺序执行的方式，CPU按任务编号依次扫描各程序段后执行I/O刷新，然后进行下一个周期的扫描。
- PLC执行到END指令，即认为程序段到此结束，后面的指令一概不执行，马上转到下一个任务程序段执行。因此，在调试复杂程序时，可把程序分成若干段，每段插入一条END指令，从头开始分段调试，调试一段就删去插入的END指令，直到整个程序调通为此。



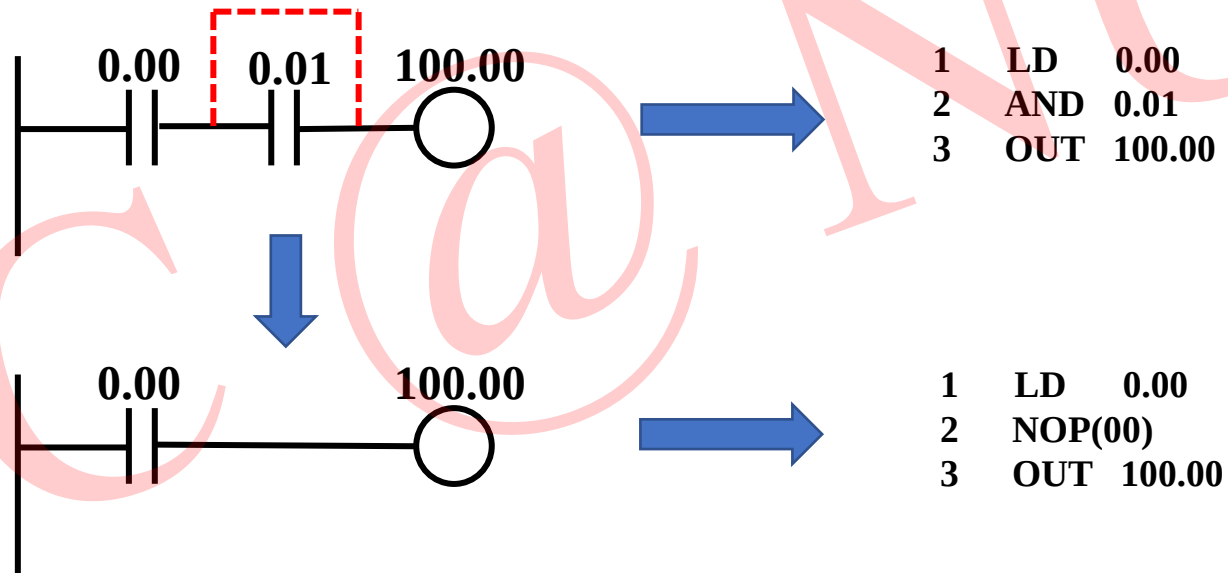
END指令的应用



## 3.3 顺序控制指令

### 2. 空操作指令—NOP (000)

功能：该指令无操作数，**无梯形图符号**，用来取消程序某一梯级（条）操作，不执行任何操作。使用该指令可占据被删除指令梯级，而**不改变程序中其它梯级的序号**。



注：修改程序时，使用NOP指令，可使步序号变更较少，便于调试程序。



## 3.3 顺序控制指令

### 3. 联锁/联锁解除指令—IL (002) /ILC (003)



IL/ILC 指令的梯形图符号

**功能：IL总是和ILC指令一起使用，用于处理梯形图中的分支电路。**

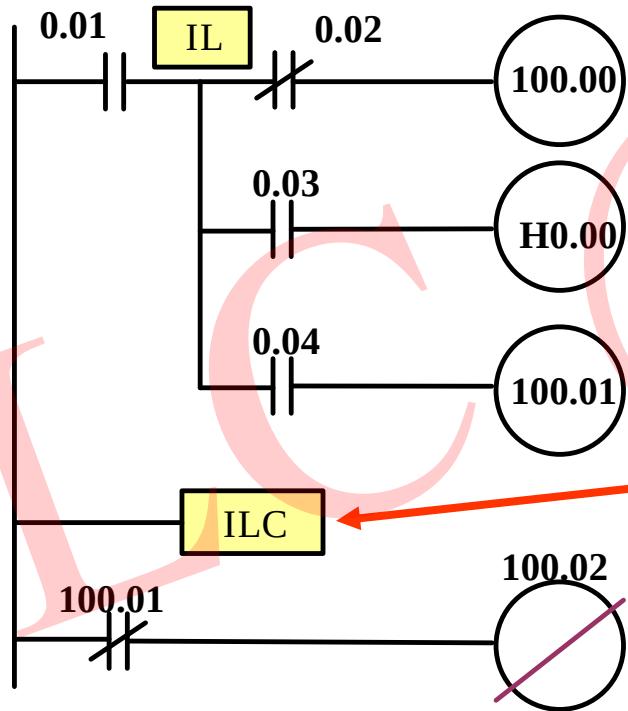
- ✓ 若执行条件为ON，则IL和ILC之间的联锁程序段正常执行；
- ✓ 执行条件为OFF，则位于IL和ILC之间的联锁程序段不执行，此时，IL和ILC之间的所有输出位为OFF；所有定时器复位；所有计数器和有保持功能指令的输出位持以前状态。



## 3.3 顺序控制指令

### 3. 联锁/联锁解除指令—IL (002) /ILC (003)

#### 分支和分支结束指令



分支指令----在分支处形成新母线

注 意:

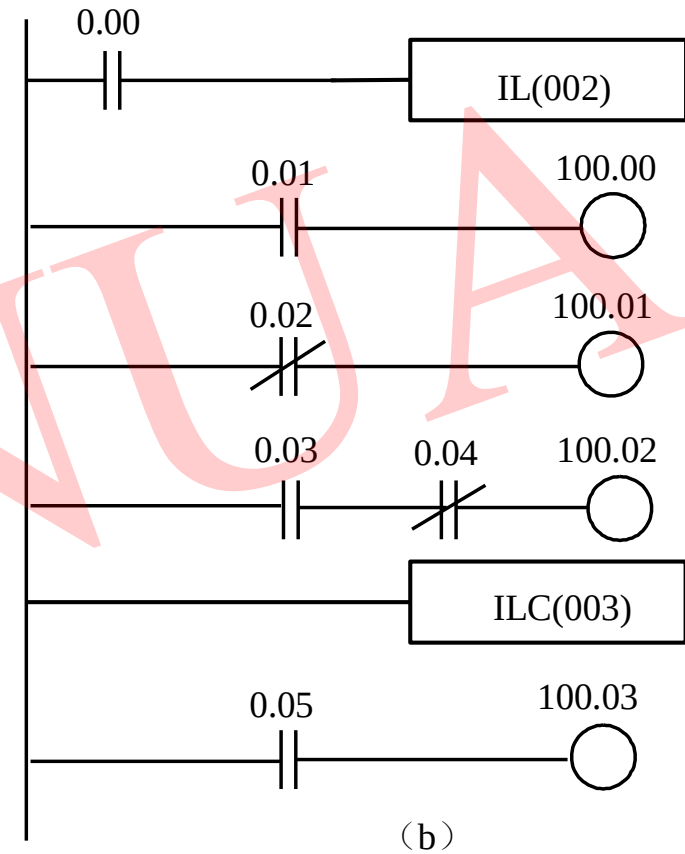
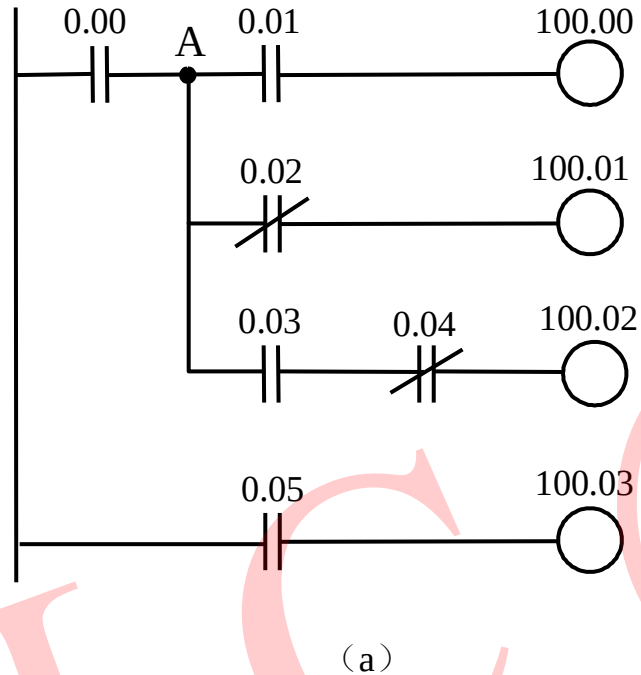
当IL指令前的逻辑关系接通时，IL和ILC之间的程序执行。当IL指令的逻辑关系断开时，IL和ILC之间的程序不执行。

分支结束指令-- 从分支处返回（消除分支）



## 3.3 顺序控制指令

### 3. 联锁/联锁解除指令—IL (002) /ILC (003)



|             |        |          |        |
|-------------|--------|----------|--------|
| LD          | 0.00   | LD       | 0.03   |
| IL(002)     |        | AND NOT  | 0.04   |
| LD          | 0.01   | OUT      | 100.02 |
| OUT         | 100.00 | ILC(003) |        |
| LD NOT 0.02 |        | LD       | 0.05   |
| OUT         | 100.01 | OUT      | 100.03 |



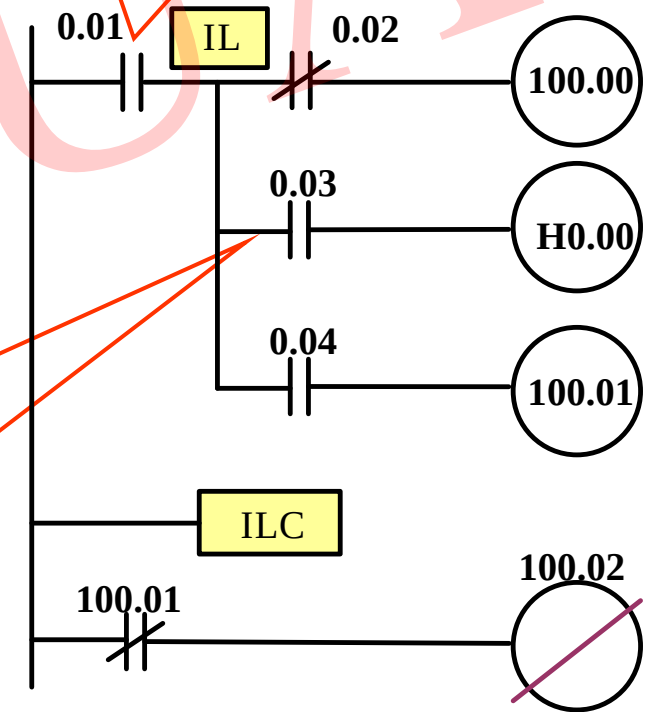
## 3.3 顺序控制指令

### 3. 联锁/联锁解除指令—IL (002) /ILC (003)

```
1 LD    0.01
2 IL
3 LD-NOT 0.02
4 OUT    100.00
5 LD      0.03
6 OUT    H0.00
7 LD      0.04
8 OUT    100.01
9 ILC
10 LD-NOT 100.01
11 OUT-NOT 100.02
```

说明：（1）执行条件是ON或OFF，对IL—ILC之间的程序段都要占用扫描时间。

当IL指令前的逻辑关系断开时，IL和ILC指令之间的程序不执行。



在IL和ILC之间的程序不执行时，它们之间的继电器状态是不同的：输出继电器、内部辅助继电器断开；定时器复位；计数器、移位寄存器、保持继电器保持当前状态。

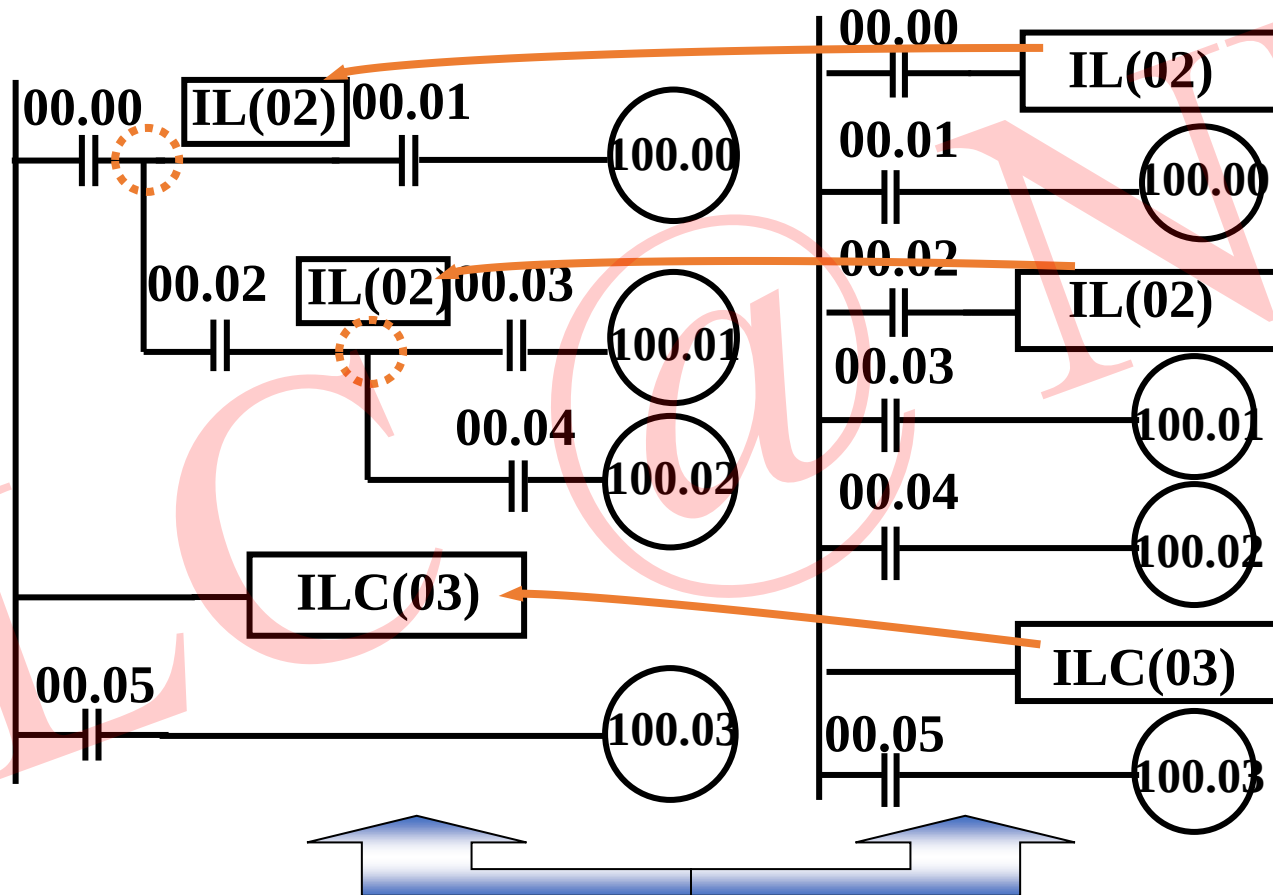




## 3.3 顺序控制指令

### 3. 联锁/联锁解除指令—IL (002) /ILC (003)

(2) IL和ILC指令可以成对使用，也可以多个IL指令配一个ILC指令，但不允许嵌套使用(如IL—IL—ILC—ILC)。如下图所示，左图联锁程序实现的功能和右图是一样的。当多个IL指令配一个ILC指令使用时，程序检查时会有出错信息显示，但不影响程序的正常执行。



功能相同

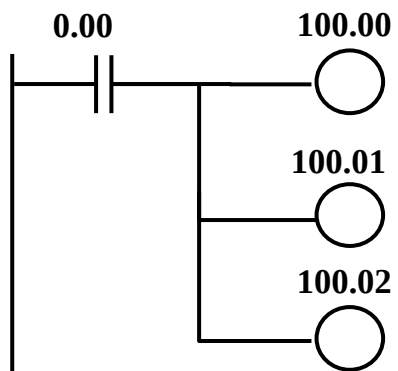
|          |        |
|----------|--------|
| LD       | 00.00  |
| IL (02)  |        |
| LD       | 00.01  |
| OUT      | 100.00 |
| LD       | 00.02  |
| IL (02)  |        |
| LD       | 00.03  |
| OUT      | 100.01 |
| LD       | 00.04  |
| OUT      | 100.02 |
| ILC (03) |        |
| LD       | 00.05  |
| OUT      | 100.03 |



## 3.3 顺序控制指令

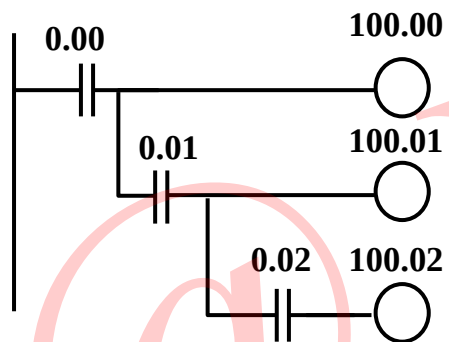
### 3. 联锁/联锁解除指令—IL (002) /ILC (003)

✓ 并联输出、连续输出和复合输出



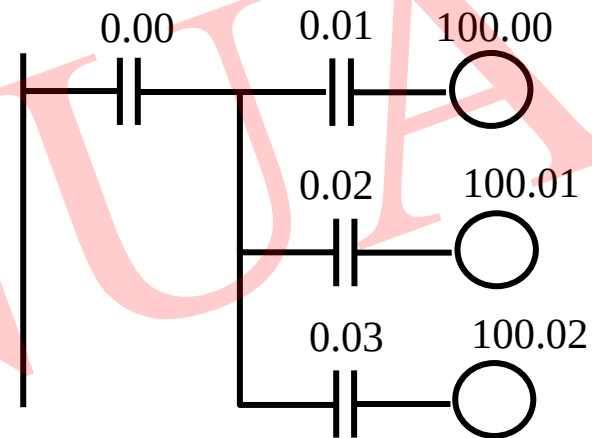
(a) 并联输出

```
LD    0.00
OUT   100.00
OUT   100.01
OUT   100.02
```



(b) 连续输出

```
LD    0.00
OUT   100.00
AND   0.01
OUT   100.01
AND   0.02
OUT   100.02
```



(c) 复合输出

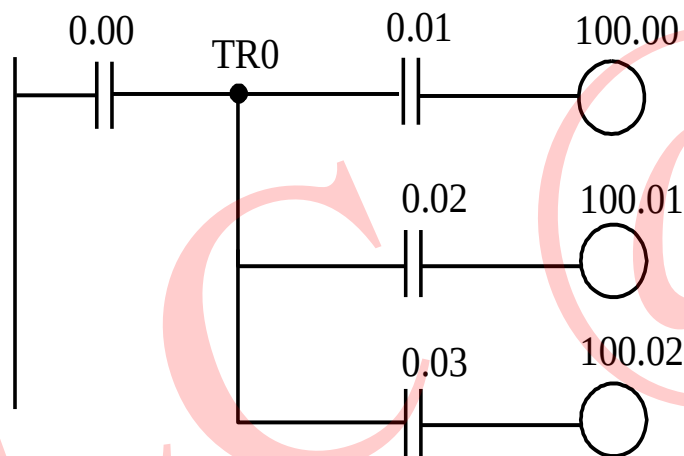
```
LD    0.00
IL(002)
LD    0.01
OUT   100.00
LD    0.02
OUT   100.01
LD    0.03
OUT   100.02
ILC(003)
```



## 3.3 顺序控制指令

### 4. 暂存继电器—TR

- 暂存继电器TR编号：TR00~TR15。
- 用来暂时存储执行结果，如果一个TR位被设置于一个分支点处，则当前分支点状态会存储在指定的TR位中。



(a) 原电路

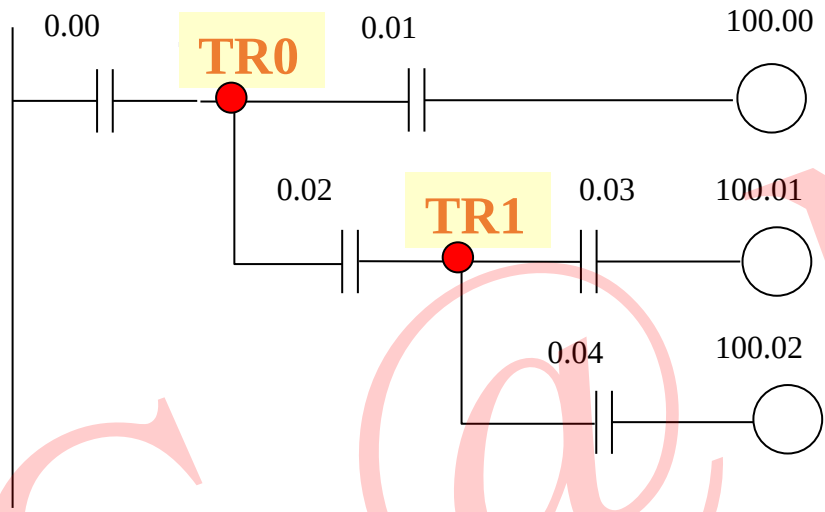
|     |        |
|-----|--------|
| LD  | 0.00   |
| OUT | TR0    |
| AND | 0.01   |
| OUT | 100.00 |
| LD  | TR0    |
| AND | 0.02   |
| OUT | 100.01 |
| LD  | TR0    |
| AND | 0.03   |
| OUT | 100.02 |

用 TR 位处理梯形图的分支



### 3.3 顺序控制指令

#### 4. 暂存继电器—TR



用TR位处理分支电路

|     |        |
|-----|--------|
| LD  | 0.00   |
| OUT | TR0    |
| AND | 0.01   |
| OUT | 100.00 |
| LD  | TR0    |
| AND | 0.02   |
| OUT | TR1    |
| AND | 0.03   |
| OUT | 100.01 |
| LD  | TR1    |
| AND | 0.04   |
| OUT | 100.02 |

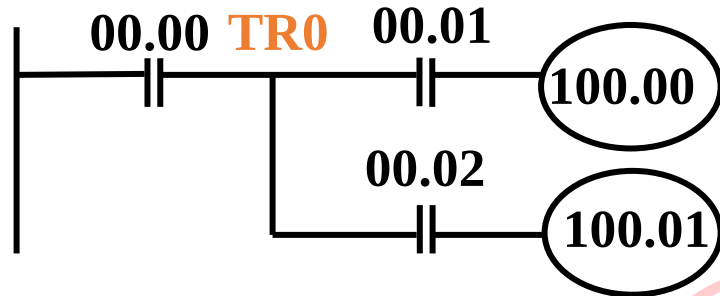
说明：

- 在同一程序段中，同一TR号不能重复使用，在不同的程序段中，同一TR号可以重复使用。
- TR不是独立的编程指令，只能和LD或OUT等基本指令一起使用。



## 3.3 顺序控制指令

### ◆ TR指令与IL/ILC指令



|     |        |
|-----|--------|
| LD  | 00.00  |
| OUT | TR0    |
| AND | 00.01  |
| OUT | 100.00 |
| LD  | TR0    |
| AND | 00.02  |
| OUT | 100.01 |

用TR处理分支

|         |        |
|---------|--------|
| LD      | 00.00  |
| IL(02)  |        |
| LD      | 00.01  |
| OUT     | 100.00 |
| LD      | 00.02  |
| OUT     | 100.01 |
| ILC(03) |        |

用IL/ILC处理分支

#### 两种处理分支方法的区别

用 TR 时：用AND指令连接下一个分支的触点

用IL/ILC时：用LD指令连接下一个分支的触点

在分支多时，用TR处理分支程序要烦琐一些。



## 3.3 顺序控制指令

### 5. 跳转/跳转结束指令—JMP (004) /JME (005)



JMP/JME 指令的梯形图符号

#### JMP /JME的用法

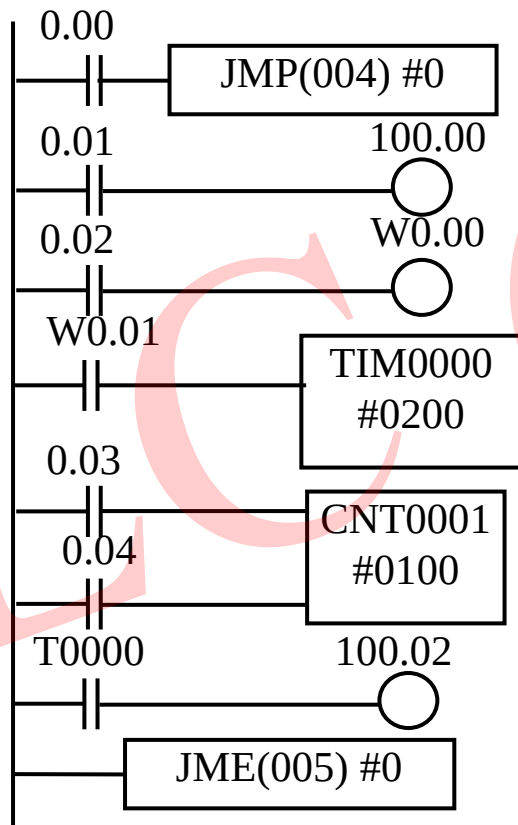
- (1) 发生跳转时，JMP N和JME N之间的程序不执行，且不占用扫描时间。
- (2) 发生跳转时所有继电器、定时器、计数器均保持跳转前的状态不变。
- (3) 同一个跳转号N只能在程序中使用一次。但当N取00时，JMP 00/JME 00可以在程序中多次使用。
- (4) 以00作为跳转号时，比其他跳转号的执行时间长。
- (5) 不同跳转号时可以嵌套使用。



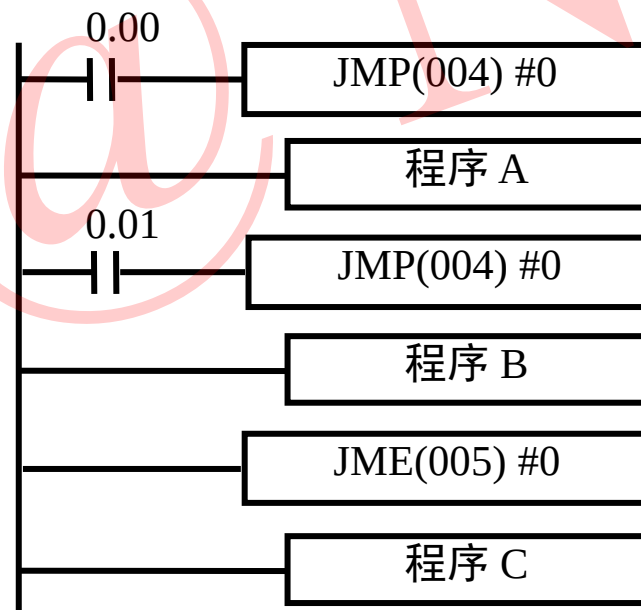
## 3.3 顺序控制指令

### 5. 跳转/跳转结束指令—JMP (004) /JME (005)

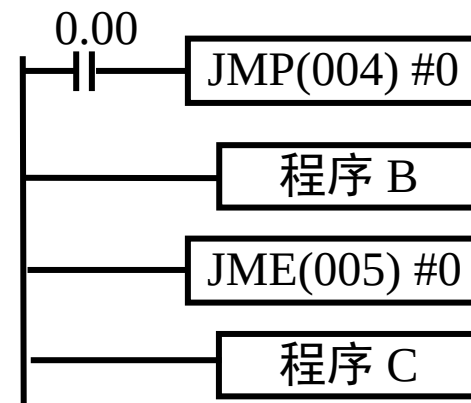
- 每个跳转号只能使用一次，即对同一个N，JMP N—JME N只能在程序中使用一次。
- 多个JMP N可以共用一个JME N，如JMP #0—JMP #0—JME #0，见图，这样使用后，在进行程序检查时会出现错误信息“JMP—JME ERR”，但程序会正常执行。



跳步的含义



多个JMP对一个JME



跳步指令的应用



## 3.3 顺序控制指令

### ✓ IL/ILC与JMP/JME

指令的共性

它们都具有控制程序执行流向的作用。

IL/ILC与JMP/JME 指令的区别

1. 对IL/ILC与JMP/JME 指令之间的程序段：

不论IL的执行条件OFF还是ON，CPU都对IL和ILC之间的程序段扫描。

当发生跳转时，JMP N和JME N之间的程序不执行，且CPU不对其扫描。





## 3.3 顺序控制指令

### IL/ILC与JMP /JME 指令的区别

#### 2. 当IL和JMP 的执行条件不满足时:

##### 对IL/ILC之间的程序段:

所有OUT指令的输出位为OFF；所有定时器都复位；KEEP指令的操作位、计数器、移位寄存器以及SET和RSET指令的操作位，都保持IL为OFF以前的状态。

##### 对JMP/JME之间的程序段:

发生跳转时所有继电器、定时器、计数器均保持跳转前的状态不变。



## 3.4 定时器/计数器指令

CP1提供如下定时计数功能：

- 普通定时器TIM/TIMX
- 高速定时器TIMH/TIMHX
- 超高速定时器TMHH/TMHHX
- 累计定时器TTIM/TTIMX
- 长定时器TIML/TIMLX
- 多输出定时器MTIM/MTIMX
- 普通计数器CNT/CNTX
- 可逆计数器CNTR/CNTRX
- 定时器/计数器复位CNR/CNRX
- 块程序的定时等待TIMW/TIMWX
- 高速定时等待TMHW/TMHWX
- 计数等待CNTW/CNTWX



# CP1提供的定时器/计数器指令

| 指令名称      |       | 助记符        | 定时/计数精度 | 定时/计数范围               | 主要特点              |
|-----------|-------|------------|---------|-----------------------|-------------------|
| 定时器       | BCD模式 | TIM        | 0.1s    | 0~9999(0~999.9s)      | 单点递减计时            |
|           | BIN模式 | TIMX(550)  |         | 0~FFFF (0~6553.5s)    |                   |
| 高速定时器     | BCD模式 | TIMH(015)  | 0.01s   | 0~9999(0~99.99s)      | 单点递减计时            |
|           | BIN模式 | TIMHX(551) |         | 0~FFFF (0~655.35s)    |                   |
| 超高速定时器    | BCD模式 | TMHH(540)  | 0.001s  | 0~9999(0~9.999s)      | 单点递减计时            |
|           | BIN模式 | TMHHX(552) |         | 0~FFFF (0~65.535s)    |                   |
| 累计定时器     | BCD模式 | TTIM(087)  | 0.1s    | 0~9999(0~999.9s)      | 单点累加计时            |
|           | BIN模式 | TTIMX(555) |         | 0~FFFF (0~6553.5s)    |                   |
| 长时间定时器    | BCD模式 | TIML(542)  | 1s      | 0~99999999(0~115天)    | 单点递减计时            |
|           | BIN模式 | TIMLX(553) |         | 0~FFFFFFFF (0~49710天) |                   |
| 多输出定时器    | BCD模式 | MTIM(543)  | 0.1s    | 0~9999<br>(0~999.9s)  | 多点累加计时            |
|           | BIN模式 | MTIMX(554) |         | 0~FFFF (0~6553.5s)    |                   |
| 计数器       | BCD模式 | CNT        | 1       | 0~9999                | 单点递减计数            |
|           | BIN模式 | CNTX(546)  |         | 0~FFFF                |                   |
| 可逆计数器     | BCD模式 | CNTR(012)  | 1或-1    | 0~9999                | 单点可逆计数            |
|           | BIN模式 | CNTRX(548) |         | 0~9999                |                   |
| 定时器/计数器复位 | BCD模式 | CNR(545)   | —       | —                     | 对指定的定时器/<br>计数器复位 |
|           | BIN模式 | CNRX(547)  | —       | —                     |                   |
| 定时等待      | BCD模式 | TIMW(813)  | 0.1s    | 0~9999(0~999.9s)      | 单点递减计数            |
|           | BIN模式 | TIMWX(816) |         | 0~FFFF (0~6553.5s)    |                   |
| 高速定时等待    | BCD模式 | TMHW(815)  | 0.01s   | 0~9999(0~99.99s)      | 单点递减计数            |
|           | BIN模式 | TMHWX(817) |         | 0~FFFF (0~655.35s)    |                   |
| 计数等待      | BCD模式 | CNTW(814)  | 1       | 0~9999                | 单点递减计数            |
|           | BIN模式 | CNTWX(818) |         | 0~FFFF                |                   |



## 3.4 定时器/计数器指令

- 定时器/计数器有两种执行模式

### BCD模式和BIN模式

- ◆ 用户程序里只能选用一种模式，两者不能混在一起使用。
- ◆ 通常默认的为BCD模式，BIN模式极大扩展了定时/计数的范围，若要使用BIN模式，需用CX-P软件在“PLC属性”里设定。
- ◆ BIN模式下，定时器/计数器指令助记符的最后一个字符为“X”。



## 3.4 定时器/计数器指令

### ➤ 使用TIM/CNT注意事项

- ① TIM/CNT使用编号T0000~4095/ C0000~4095 。
- ② TIM/CNT的操作数可以是常数，也可以是通道，但常数或通道内容必须是BCD数。
- ③ TIM/CNT的设定值是通道时，通道内容改变、设定值即改变。但必须在TIM的输入条件断开、CNT复位后，下一次定时/计数才执行新设定值。
- ④ 计数器有保持功能，而定时器没有此功能。



### 3.4.1 定时器类指令

## 1. 定时器TIM/ TIMH

## TIMH——高速定时器 定时时间为0~99.99 s

格式:  $\begin{matrix} \text{TIM} & \text{N} \\ & \text{SV} \end{matrix}$  符号  $\begin{matrix} \text{TIM} & \text{N} \\ & \text{SV} \end{matrix}$

**N: 000~4095** (TIM与TIMH共用这些编号)

**SV: 0~9999**    **单位: TIM 为0.1 s; TIMH 为0.01 s**

设定值SV无论是常数还是通道内的内容，都必须是BCD数。



## 3.4 定时器/计数器指令

### ★ 定时器的功能

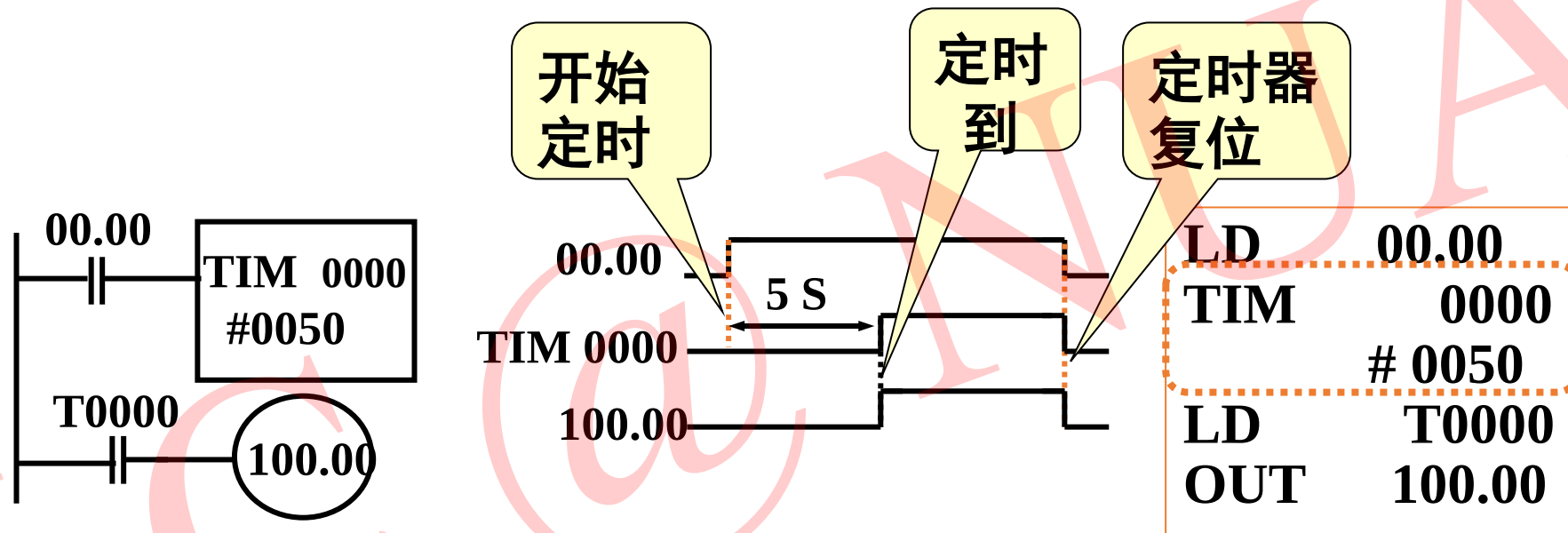
- ① 当执行条件ON时开始定时，定时过程中执行条件要保持ON。
- ② 定时时间到，定时器ON、其所属触点动作。此后只要执行条件保持ON，其ON状态保持。
- ③ 当扫描时间 $T_s > 0.1$ 秒时，TIM会不准确；  
当 $T_s > 0.01$ 秒时，TIMH会不准确。
- ④ 定时器ON后，若执行条件OFF，定时器复位。
- ⑤ 定时器ON后，若PLC断电，定时器复位。

当定时器复位时，其当前值为设定值。定时器没有断电保持功能



## 3.4 定时器/计数器指令

### ★定时器TIM、TIMH的用法



TIM定时时间:  $50 \times 0.1 = 5s$

对TIMH计算:  $50 \times 0.01 = 0.5s$





## 3.4 定时器/计数器指令

### 定时器定时功能例(1)

### 分析程序对线圈100.00的控制

00.00对应启动按钮、00.01对应停车按钮。

按下启动按钮 → 线圈00.00ON→

触点00.00闭合→ 线圈W00.00ON→

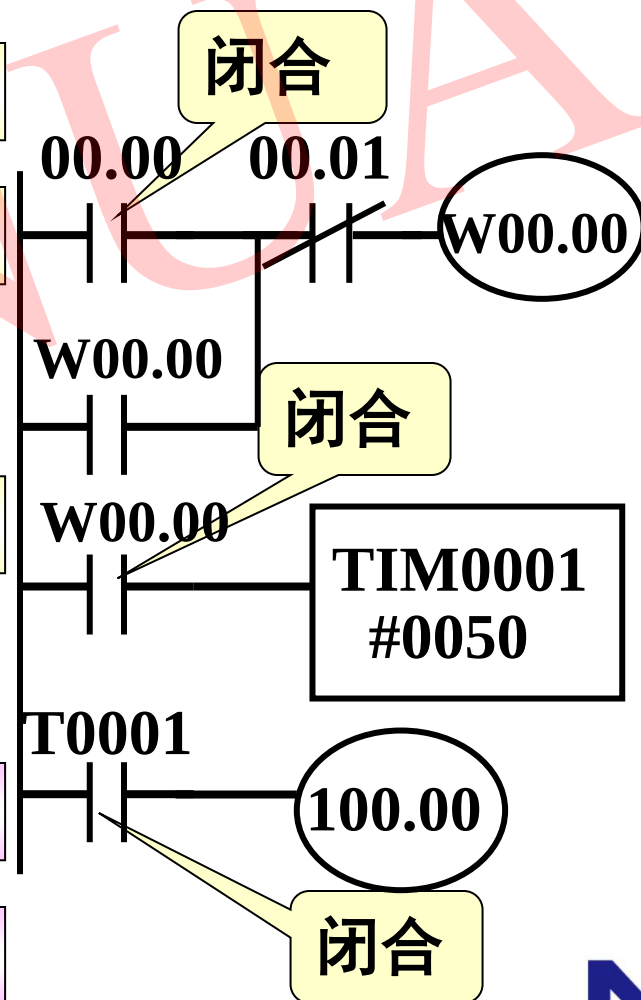
触点W00.00闭合→

TIM0001开始定时→ 经过 5s→

TIM0001 ON →

触点T0001闭合→ 线圈100.00 ON

自按下启动按钮5s后线圈100.00 ON





## 3.4 定时器/计数器指令

只要TIM0001 ON

线圈10000 一直ON

线圈100.00 ON期间按下停车按钮 →

线圈00.01通电 →

触点00.01断开 →

线圈W00.00断电 →

触点W00.00断开 →

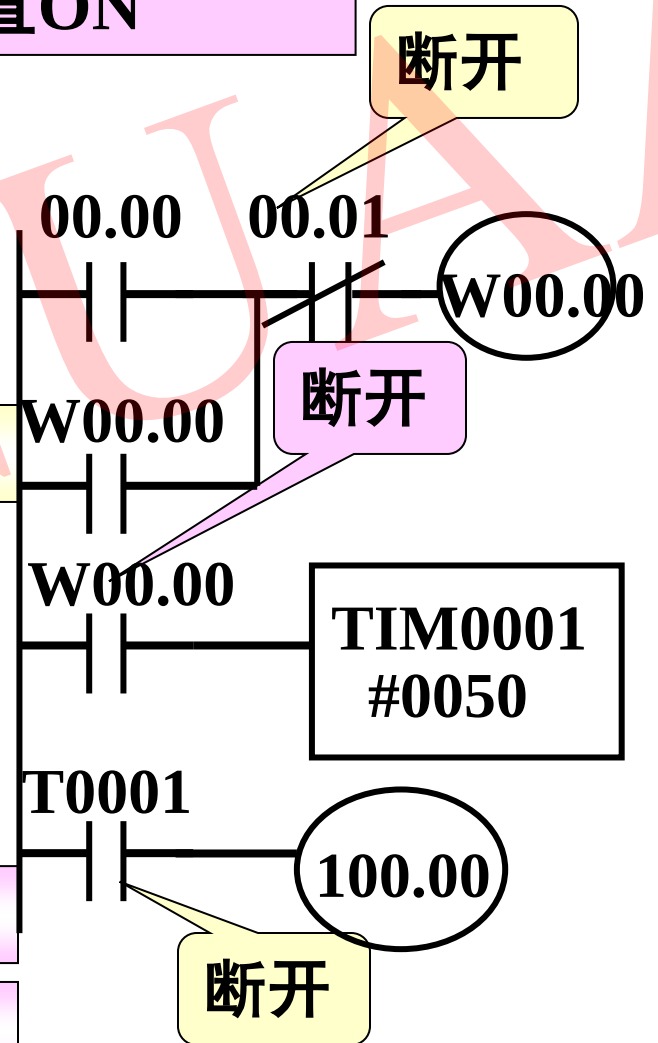
TIM0001 复位 →

触点T0001断开 →

线圈100.00 OFF

自按下启动按钮 5s后线圈100.00 ON

按下停车按钮, 线圈100.00 OFF





## 3.4 定时器/计数器指令

### 定时器定时功能例(2)

分析程序对100.00的控制作用。

设H00.00中数据为0100，TIM0001的定时值为10 秒。

00.00闭合→W00.00接通

TIM0001开始定时→

10秒到→TIM0001 ON→

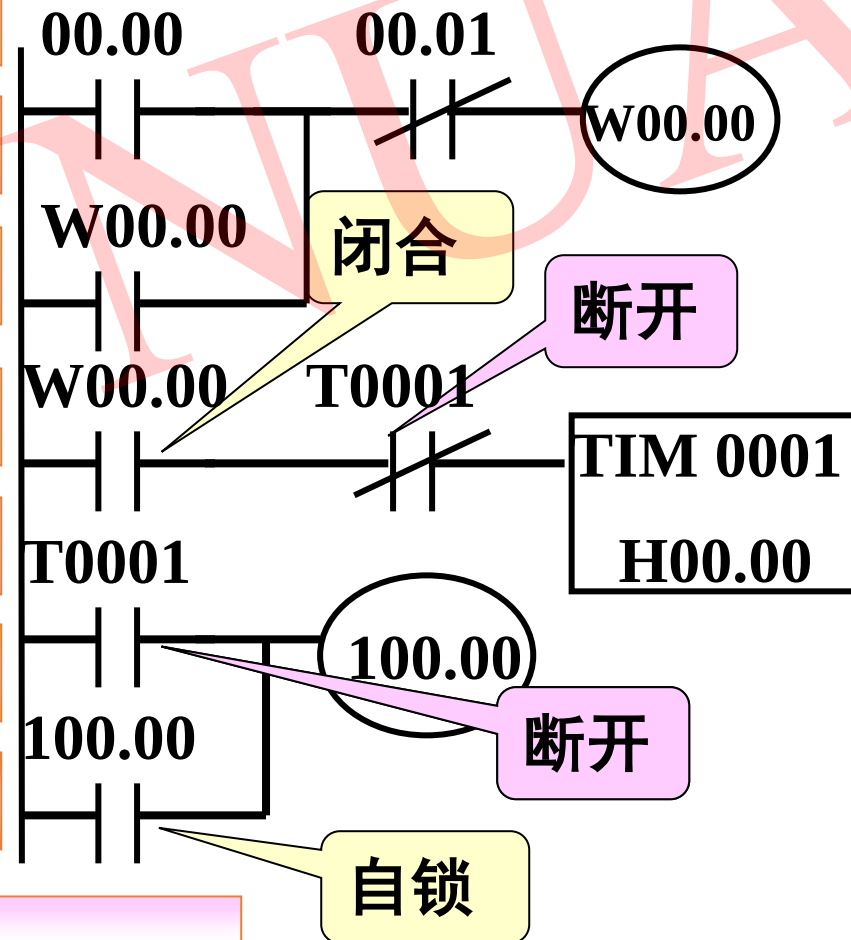
常开触点T0001 ON→

线圈100.00 ON

常闭触点T0001 OFF

TIM0001 自复位！

欲保持100.00 ON——需加自锁





## 3.4 定时器/计数器指令

TIM0001自复位后，即开始下一轮定时。

在100.00 ON期间，若  
线圈00.01 ON→

触点00.01断开→

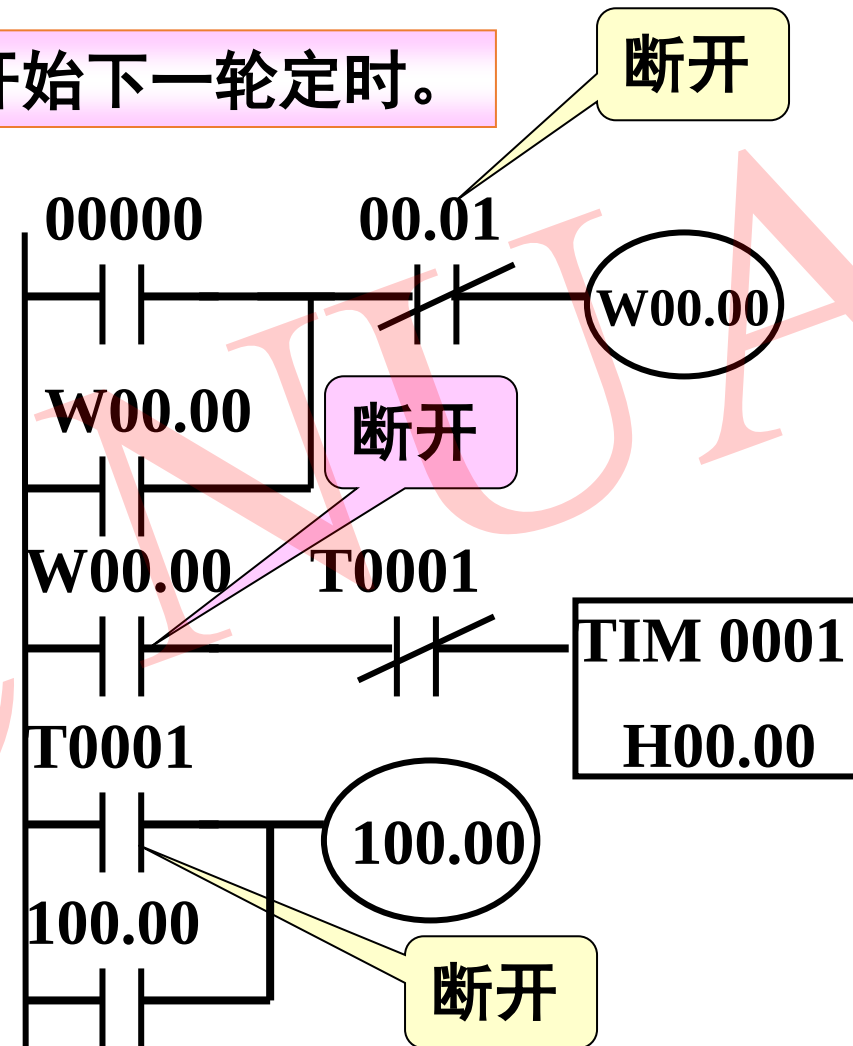
线圈W00.00断电→

TIM0001 复位→

线圈100.00 自锁

当改变通道H00.00中的内容时，  
TIM的设定值即改变！

但需TIM复位后、下一次定时才执行新设定值





## 3.4 定时器/计数器指令

### ★ 定时器定时时间的扩展

分析程序对100.00的控制作用。

自00.00 ON→线圈W00.00 ON

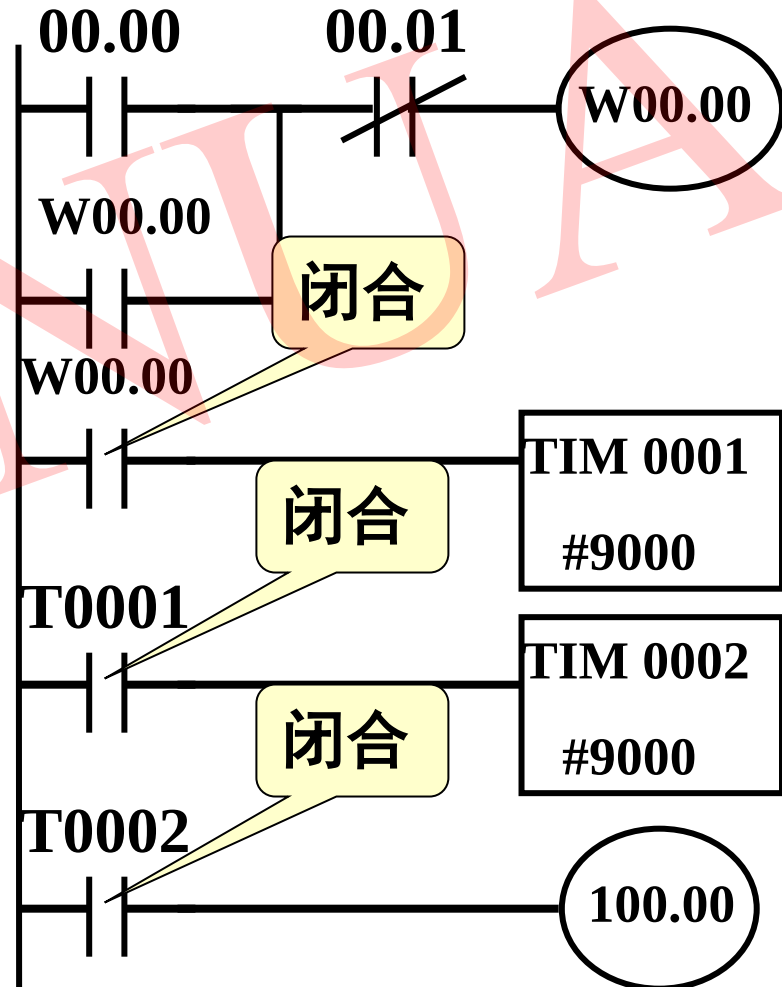
TIM0001开始定时→

900秒到→TIM0001 ON→

TIM0002开始定时→

900秒到→TIM0002 ON→

线圈100.00 ON



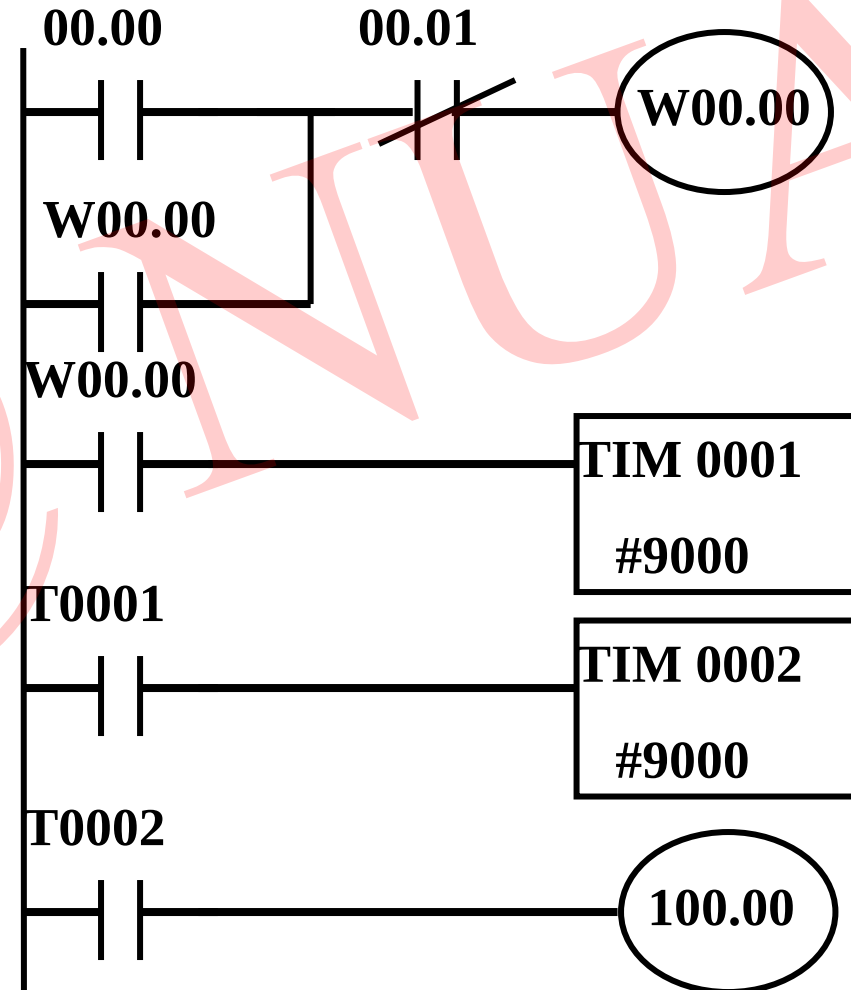
定时器级联使用—— $SV = SV1 + SV2$



### 3.4 定时器/计数器指令

练习：写出梯形图的语句表。

|         |                |
|---------|----------------|
| LD      | 00.00          |
| OR      | W00.00         |
| AND NOT | 00.01          |
| OUT     | W00.00         |
| LD      | W00.00         |
| TIM     | 0001<br>#9000  |
| LD      | T0001          |
| TIM     | 0002<br># 9000 |
| LD      | T0002          |
| OUT     | 100.00         |





### 3.4 定时器/计数器指令

00.00

IL(02)

TIM 0000  
#0600

T0000

100.00

T0000

100.01

ILC (03)

|          |        |
|----------|--------|
| LD       | 00.00  |
| IL (02)  |        |
| TIM      | 0000   |
|          | #0600  |
| LD       | T0000  |
| OUT      | 100.00 |
| LD NOT   | T0000  |
| OUT      | 100.01 |
| ILC (03) |        |

## 接通延时ON

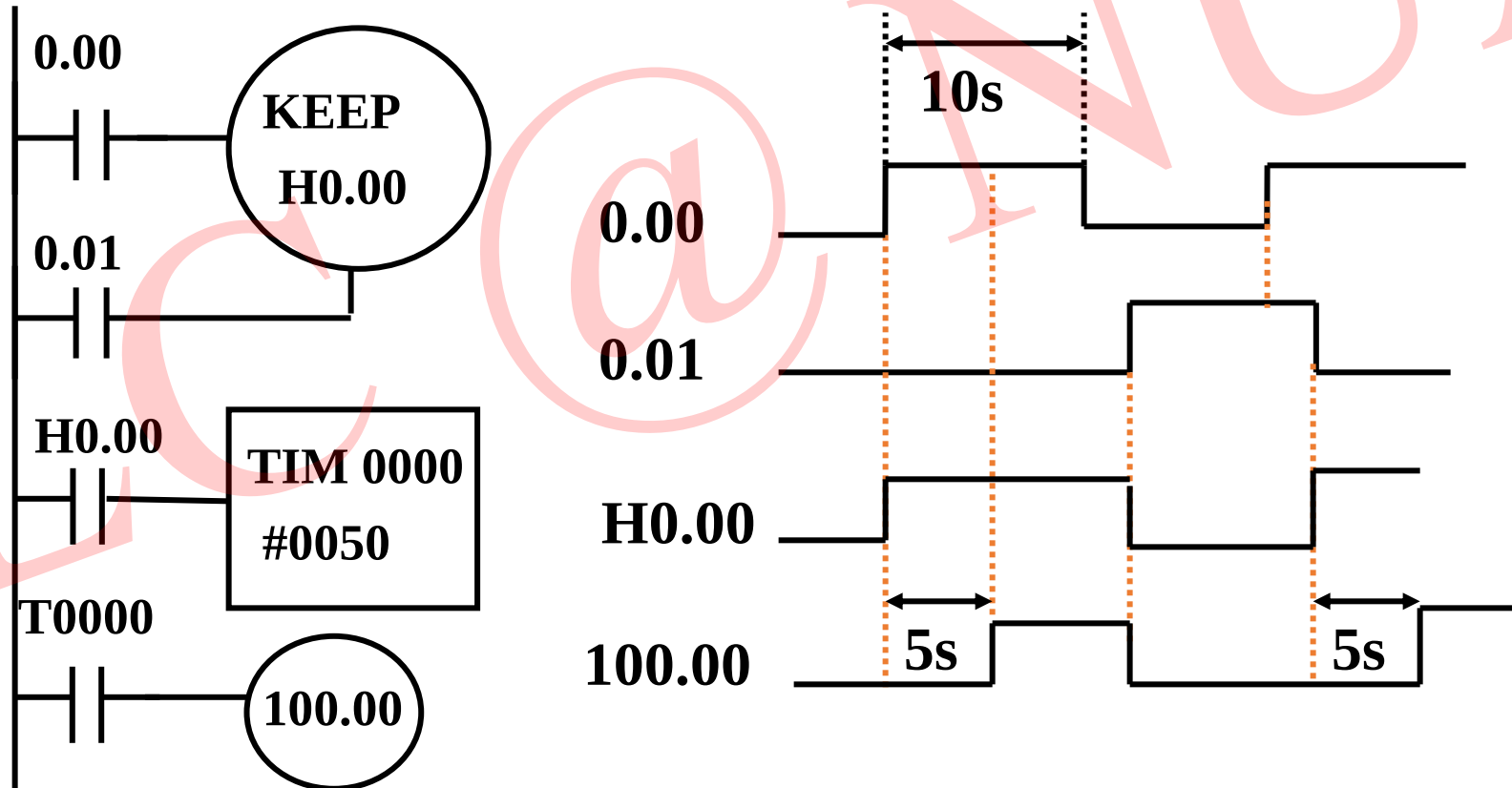
## 接通延时OFF



## 3.4 定时器/计数器指令

### 使用KEEP、TIM指令举例

根据0.00和0.01的波形，画出程序的工作时序。







## 3.4 定时器/计数器指令

- TIMX (550)

**TIMX**的功能与TIM相同，区别是设定值SV为十六制数，取值范围是0000~FFFF，定时范围是0~6553.5s。与TIM相比，TIMX的定时范围大大扩展了。



## 3.4 定时器/计数器指令

### 3.4.2 计数器类指令

#### 1. 计数器指令CNT/CNTX(546)



N: 0000~4095

SV: BCD 0 ~ 9999

CP: 计数输入端（每输入一个脉冲，CNT计一个数）

R: 复位端（R端ON时计数器复位，即CNT清零）



## 3.4 定时器/计数器指令

### ★ 计数器的用法

- ① 计数器的SV值可以是常数，也可以是通道号。
- ② 计数达到设定值时CNT ON、其所属接点动作。
- ③ CNT ON 后，此后输入的计数脉冲无效。
- ④ 计数器ON后，只要不清零，CNT将保持ON。
- ⑤ CNT ON后，若输入条件OFF，或PLC断电，计数器当前值能保持。

当计数器复位时，其当前值为设定值。计数器具有断电保持功能。

CNTX的功能与CNT相同，区别是设定值SV为十六进制数，取值范围是0000~FFFF。

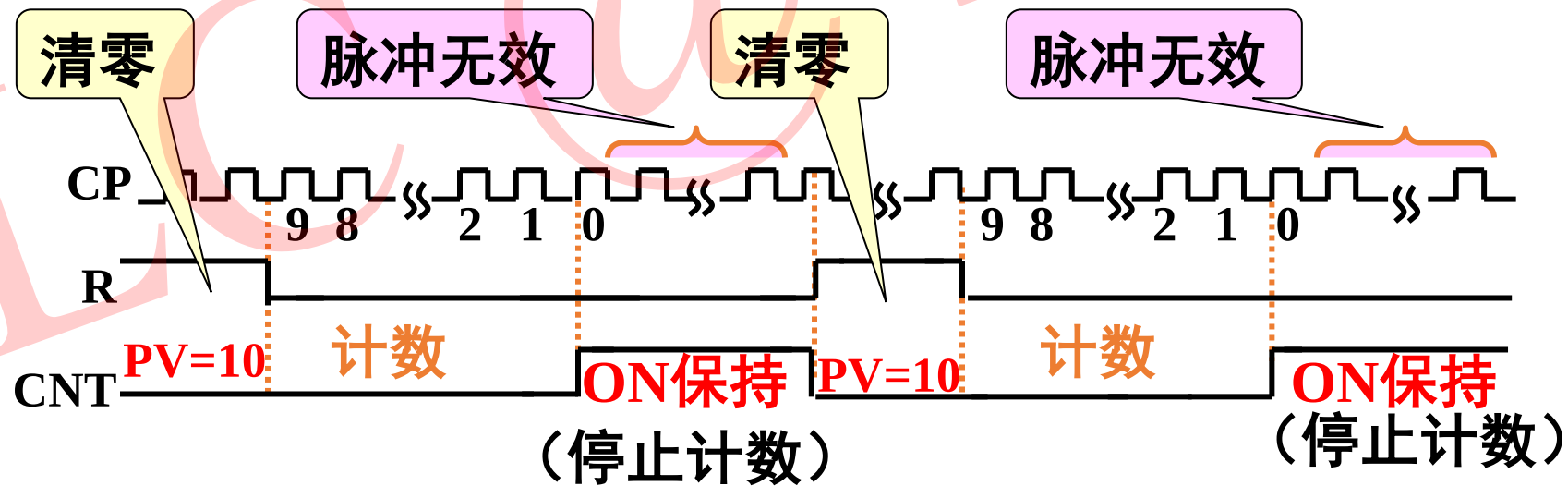


## 3.4 定时器/计数器指令

### ★ 计数器的工作时序

CNT是减计数器，每输入一个计数脉冲，由SV值减1。

SV=10 时 CNT的工作波形





## 3.4 定时器/计数器指令

### ★ CNT的计数功能举例

分析程序对100.00的控制作用。

00.00每通断一次，向CNT0000输入一个计数脉冲。

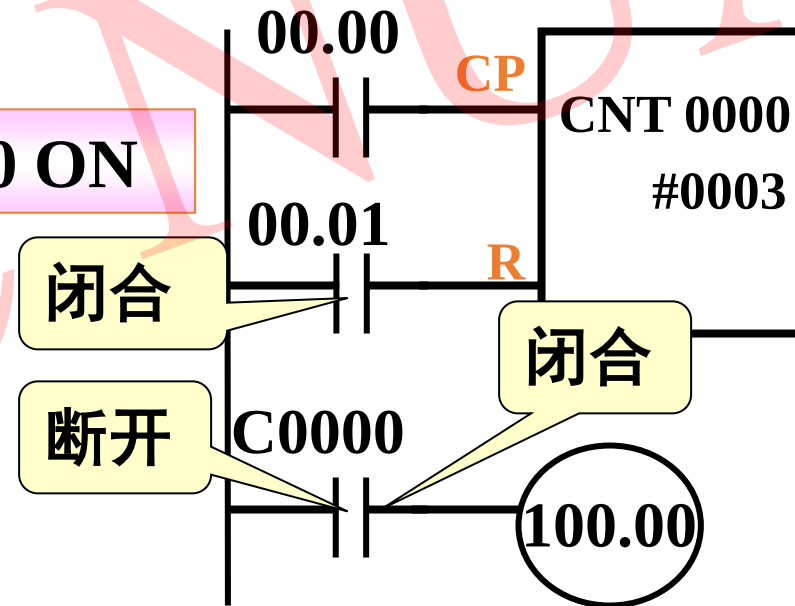
00.00通断3次→

CNT0000 ON→ 线圈100.00 ON

此后若触点00.01 ON→

CNT0000 复位→

线圈100.00 OFF



若将#0003换成020，则计数值是通道020中的数据。

CNT0000 断电再复电时，能保持断电前的当前值。

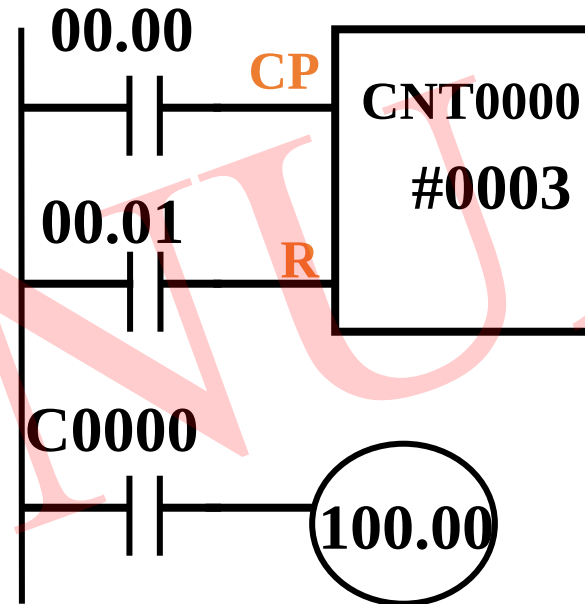


## 3.4 定时器/计数器指令

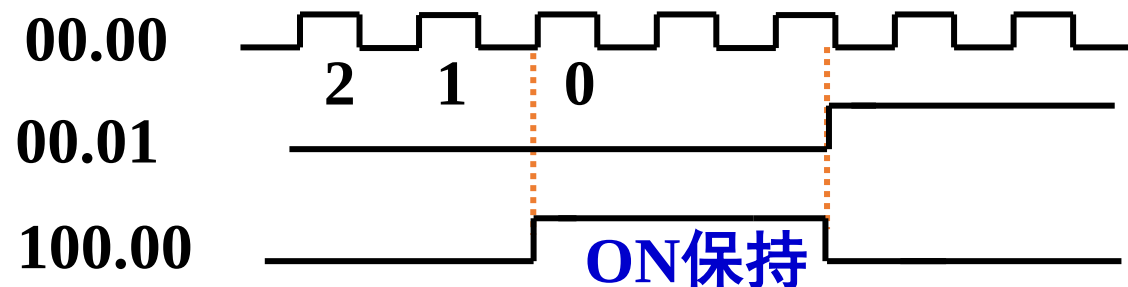
### 写出梯形图程序的语句表

SV为  
常数

|     |        |
|-----|--------|
| LD  | 00.00  |
| LD  | 00.01  |
| CNT | 0000   |
|     | #0003  |
| LD  | C0000  |
| OUT | 100.00 |



### CNT0000计数器的工作时序





## 3.4 定时器/计数器指令

### ★ 计数器容量的扩展

CNT0000采用自清零。

上电之初P\_First\_Cycle为两个计数器清零。

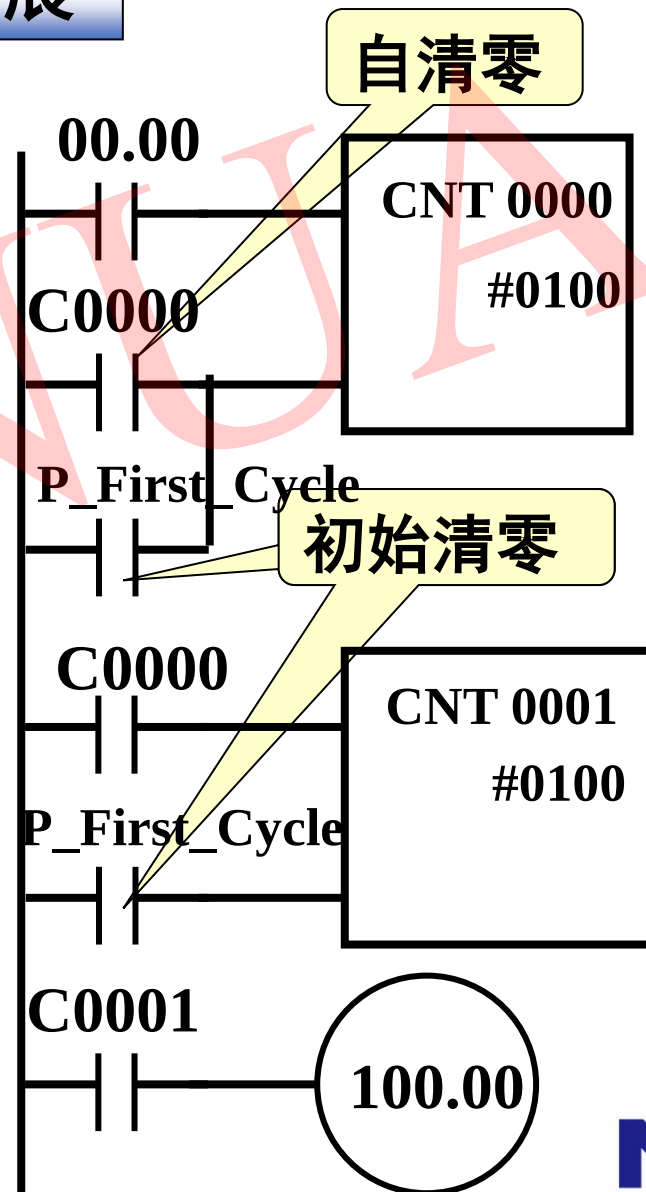
每当 CNT0000计数设定值到：

- 向CNT0001输入一个计数脉冲。
- 自清零、并开始下一轮计数。

当 CNT0001计数到，100.00 ON。

00.00 通断了100.00 次时100.00 ON

★计数器级联 $SV = SV1 \times SV2$





## 3.4 定时器/计数器指令

### ★计数器的定时器功能

分析程序对100.00的控制作用。

P\_1s能产生周期为1秒的脉冲

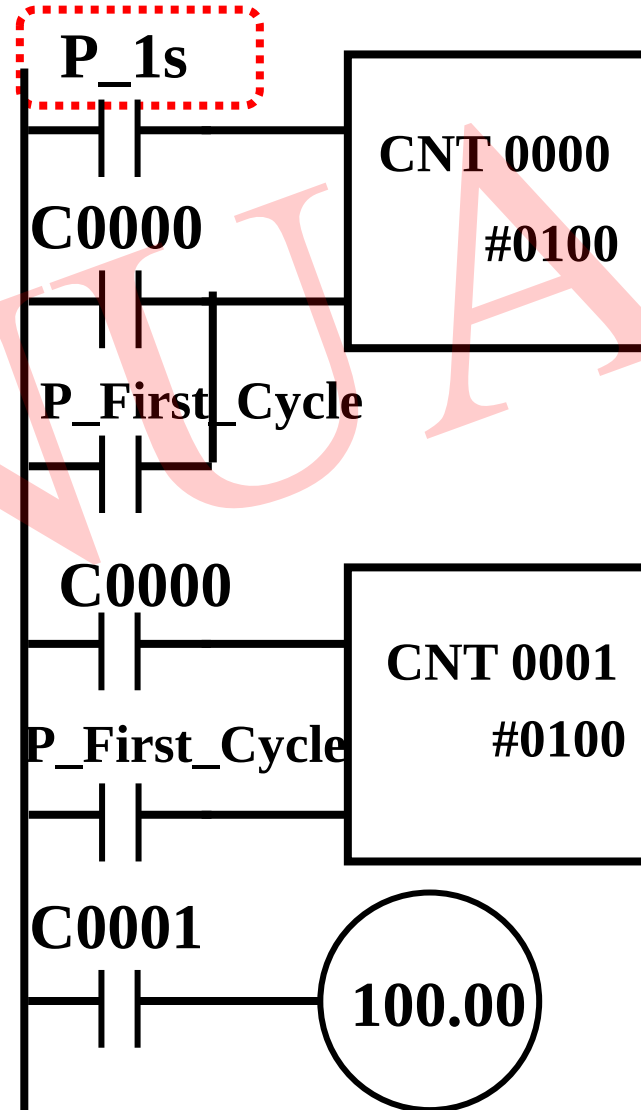
CNT0000完成计数100所经历的时间是100秒。

CNT0001完成计数100所经历的时间是100.00秒。

经历100.00秒后100.00 ON

计数器作为定时器使用

用计数器作定时器时，该定时器具有保持功能。







### 3.4 定时器/计数器指令

#### 2.可逆计数器指令CNTR(012)/CNTRX(548)

格式: CNTR(012)

N

SV

加计数输入

减计数输入

复位端

ACP

SCP

R

CNTR N

SV

N: 编号 (与CNT共用0000~4095)

SV: 设定值 (BCD 0 ~9999)



## 3.4 定时器/计数器指令

### ★ 可逆计数器的功能

① 从ACP或SCP输入计数脉冲，可组成加或减计数器。

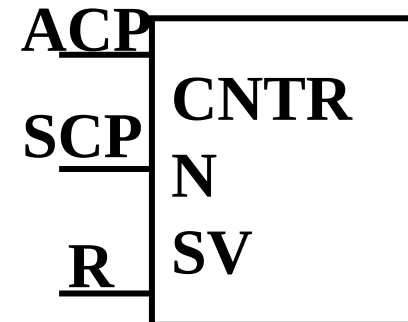
② 从ACP和SCP端同时输入计数脉冲，计数器不计数。

③ 计数器复位时，不论是加还是减计数，其PV均为0。

④ 加/减计数有进/借位时，输出ON一个计数脉冲周期。

⑤ 可逆计数器可作为循环计数器。

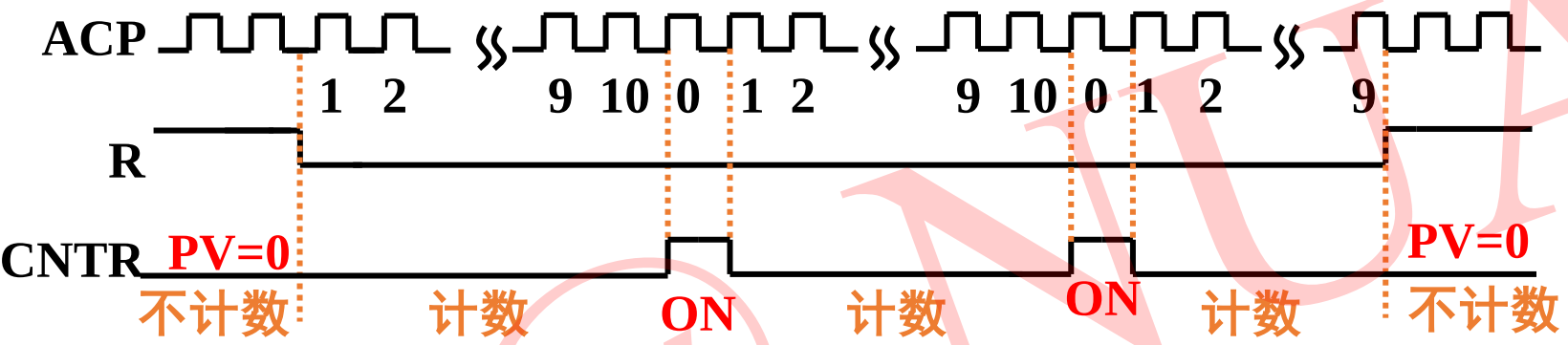
计数器达到设定值ON时，若再来一个计数脉冲，CNTR OFF、且开始下一个循环的计数。



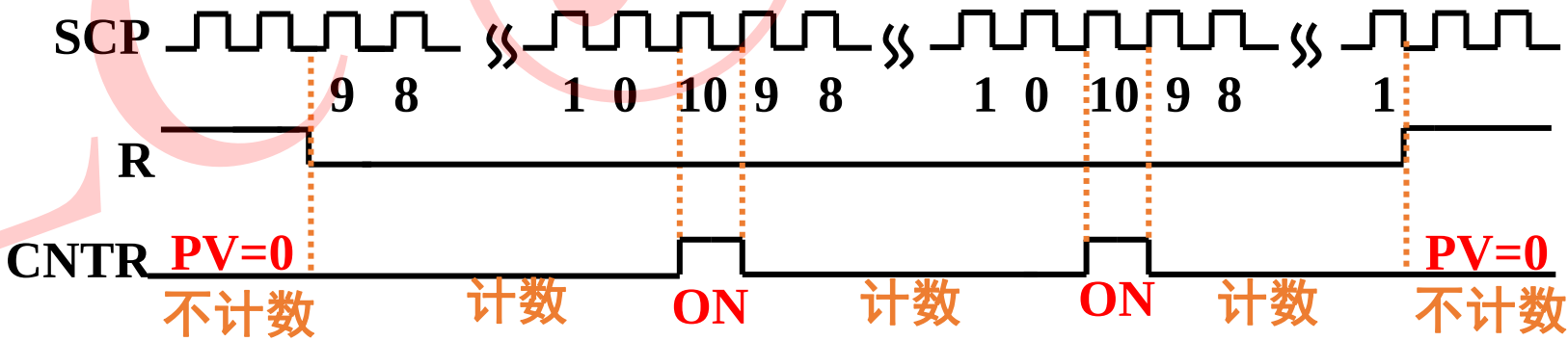


### 3.4 定时器/计数器指令

#### ★ 可逆计数器的工作时序



#### SCP=0 SV=10 加法计数时序



#### ACP=0 SV=10 减法计数时序



## 3.4 定时器/计数器指令

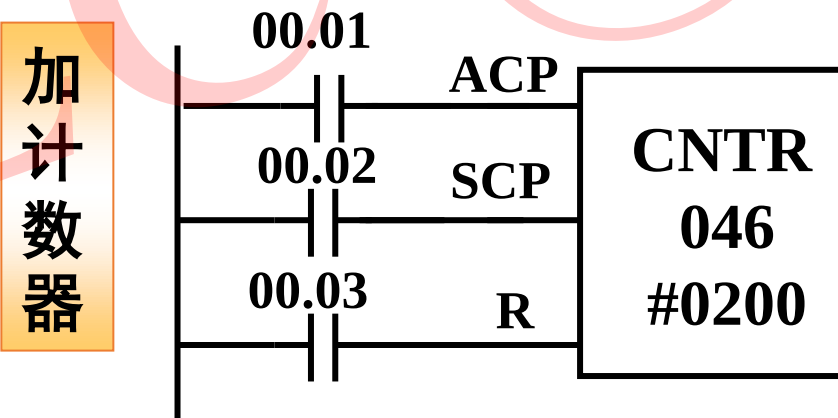
### ★ 可逆计数器的计数功能举例

设00.03 OFF、00.02 OFF，00.01输入计数脉冲。

00.01每输入一个计数脉冲，CNTR 046的当前值加1。

当PV=0200时，再输入一个计数脉冲，PV值变为0000（进位），同时CNTR 046变为ON。

此后再来一个计数脉冲时，PV值变为0001，CNTR 046为OFF，且开始下一个循环的计数。



|          |       |
|----------|-------|
| LD       | 00.01 |
| LD       | 00.02 |
| LD       | 00.03 |
| CNTR(12) | 046   |
|          | #0200 |



## 3.4 定时器/计数器指令

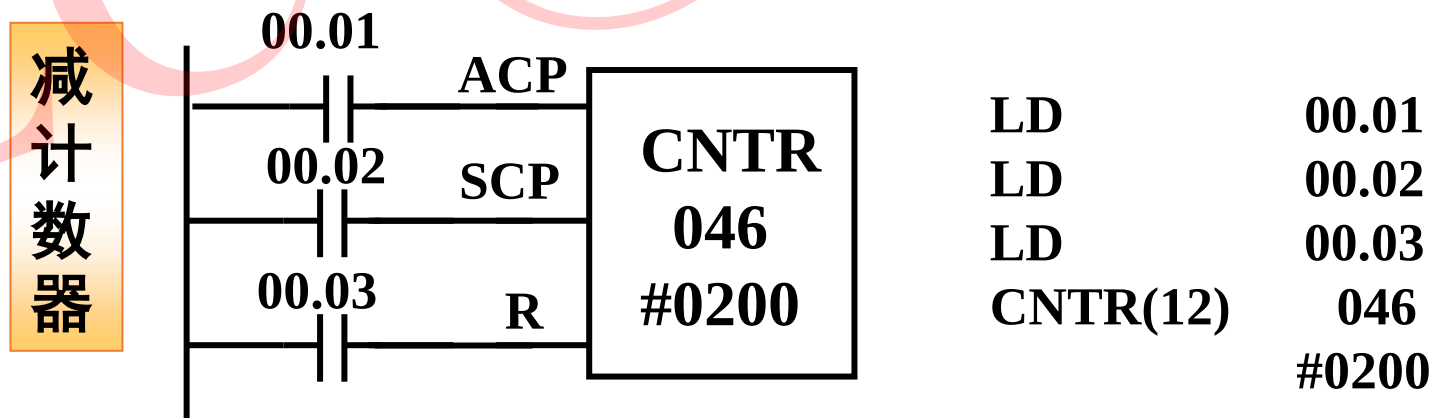
### ★ 可逆计数器的计数功能举例

设00.03 OFF、00.01 OFF，00.02输入计数脉冲。

00.02每输入一个计数脉冲，CNTR 046的当前值减1。

当PV=0000时，再输入一个计数脉冲，PV值变为0200（借位），同时CNTR 046变为ON。

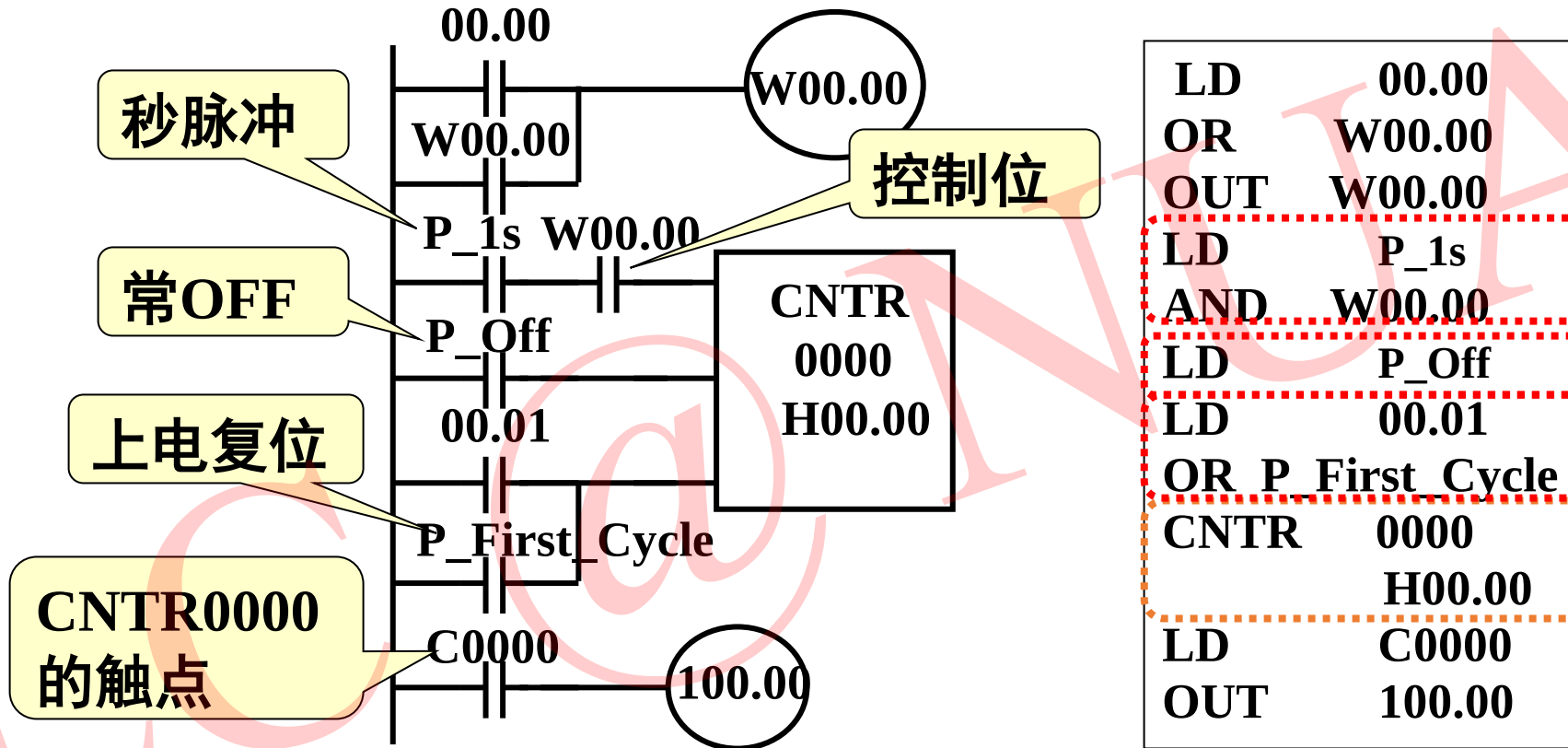
此后再来一个计数脉冲时，PV=0199，CNTR 046为OFF，且开始下一个循环的计数。





## 3.4 定时器/计数器指令

### ★ 可逆计数器的定时器功能举例



CNTR000 是加计数器

由于计数脉冲是秒脉冲，所以可作定时器。



## 3.4 定时器/计数器指令

设00001为OFF，H0000中数据是500，分析功能。

PLC上电，计数器即复位。

自00.00 ON 开始，CNTR0000开始计数→

计数达500 CNTR0000 ON →

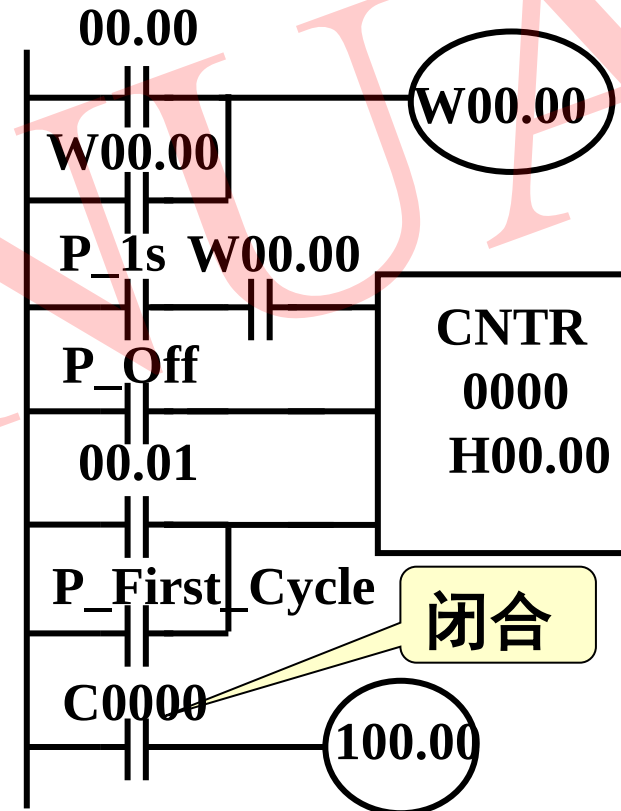
CNTR0000 的触点ON →

线圈100.00 ON

即经过500秒线圈100.00 ON

再过1秒，线圈100.00 又OFF

CNTR0000开始下一循环计数



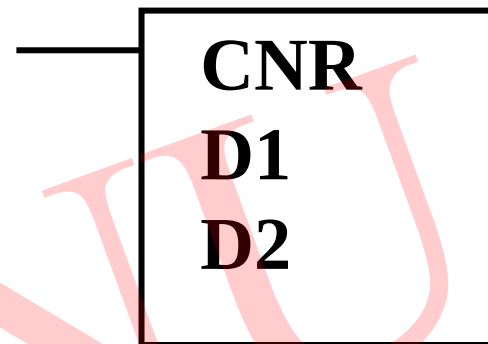
线圈100.00每过 500 秒 ON 1秒



## 3.4 定时器/计数器指令

### 3.4.3 定时器/计数器复位指令CNR/CNTX

格式:   CNR  
          D1  
          D2



D1: 定时器/计数器首编号 (T/C0000~4095)

D2: 定时器/计数器首编号 (T/C0000~4095)

D1、D2 必须在同一区域, 并且 $D1 \leq D2$

功能: D1到D2的定时器/计数器完成标志复位, 当前值置PV为最大值9999。

CNRX的功能与CNR相同, 区别是它将定时器/计数器的PV值置为最大值FFFF。





## 3.4 定时器/计数器指令

### 定时计数类指令小结

1. 本节介绍的应用指令都经常使用，要牢记其格式、梯形图、操作数范围及功能。
2. 注意定时计数类指令的编号方法、量程和定时精度。
3. 注意使用定时计数类指令的起始时间点、到时时间点、复位时间点。
4. 学会用时序图描述定时器、计数器工作状态的方法。
5. 学会定时器和计数器容量扩展的编程方法。